

- **Climatización y Calidad del Aire:** Tal como se aprecia en el diagrama (Unidad de Aire Acondicionado / ACU), el sistema mantendrá una temperatura constante de 22°C y una humedad relativa del 45-50%, niveles óptimos para el hardware. Se utilizará un sistema de "pasillo frío / pasillo caliente" para no desperdiciar energía. El aire se filtrará con sistemas HEPA para evitar la acumulación de polvo.
- **Cableado, Suelo y Techo Técnico:** Se ha diseñado un suelo técnico elevado por donde pasará todo el cableado eléctrico y el flujo de aire frío impulsado desde abajo. También cuenta con un techo técnico (falso techo) por donde pasará el cableado de datos (fibra y cobre) ordenado en bandejas, y las tuberías del sistema anti-incendios de gas.

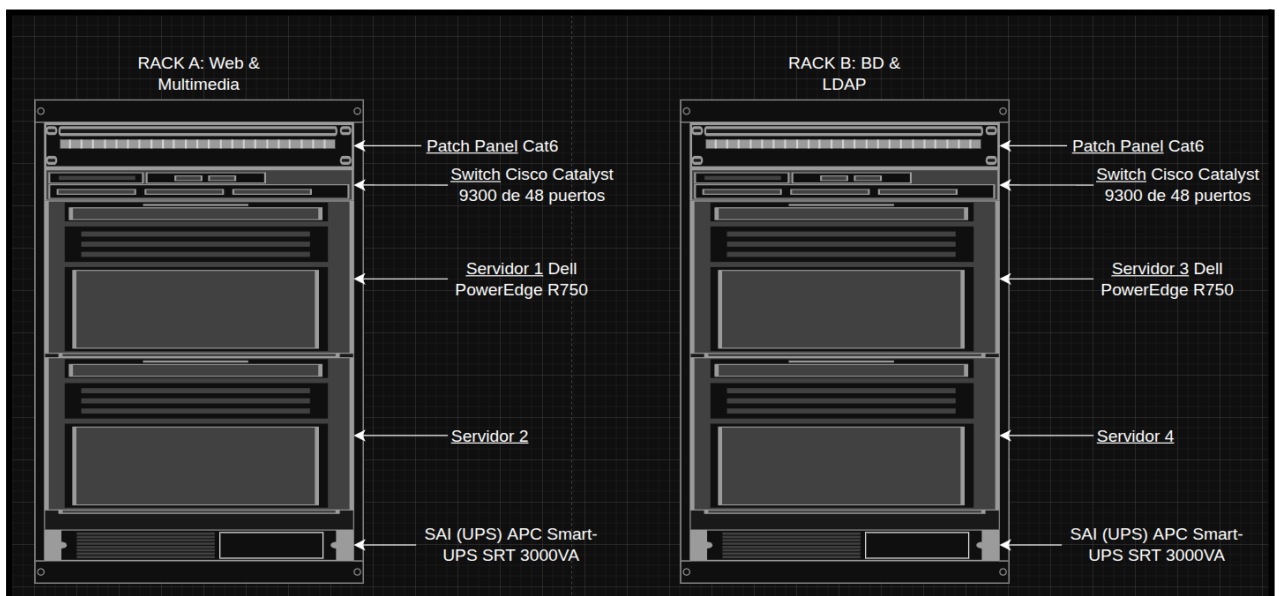
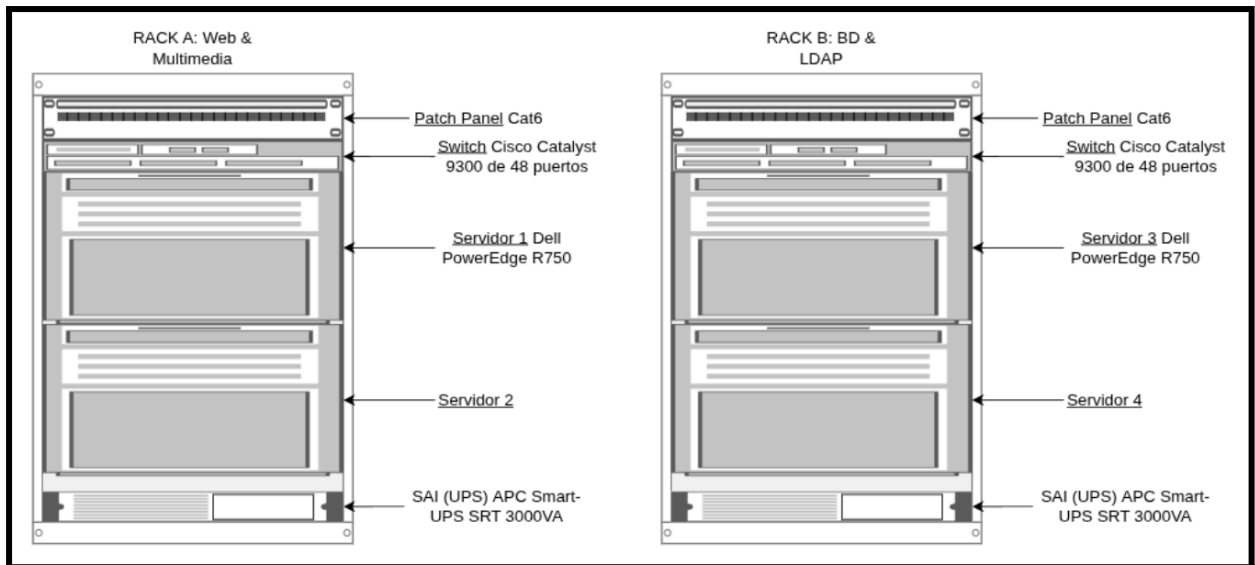
- **Mesuras de ocultación:** La puerta de acceso será totalmente opaca, de acero reforzado, y no tendrá ningún rótulo que indique "CPD" o "Servidores". Simulará ser un cuarto de mantenimiento general (Telecom Room) para evitar llamar la atención.

1.2 Infraestructura IT

Se ha diseñado una infraestructura de alta disponibilidad distribuida en dos armarios **Rack de 42U** cerrados (identificados como RACK A y RACK B). La disposición de los equipos se ha realizado siguiendo criterios de eficiencia térmica, seguridad física y gestión lógica de servicios.

Equipamiento de Red y Servidores:

- **Servidores (Dell PowerEdge R750):** Se han adquirido 4 unidades físicas de alto rendimiento distribuidas de la siguiente forma:
 - **Servidor 1 (RACK A):** Nodo de servicios integrados que aloja la **página Web**, el servidor **SFTP** y la plataforma de **Streaming Multimedia** (audio y vídeo).
 - **Servidor 2 (RACK A):** Servidor dedicado exclusivamente a la gestión de identidades y autenticación centralizada (**LDAP**).
 - **Servidor 3 (RACK B):** Nodo de auditoría y seguridad encargado de la centralización de **Logs** de toda la infraestructura.
 - **Servidor 4 (RACK B):** Servidor dedicado al motor de **Base de Datos** relacional para garantizar alta velocidad de consulta.
- **Switches (Cisco Catalyst 9300 - 48 puertos):** Se instalan 2 unidades (una por rack) para garantizar la redundancia total. Aunque el número de servidores es reducido, se han seleccionado modelos de 48 puertos para permitir la **escalabilidad futura** de la empresa y facilitar el **stacking (apilado)** de switches. Esto permite conectar cada servidor a ambos switches simultáneamente, asegurando que si un switch falla, los servicios sigan operativos.
- **Patch Panels (CAT6A):** 2 paneles de 24 puertos ubicados en la parte superior de cada rack. Su función es centralizar el cableado estructurado, facilitando el mantenimiento y garantizando la certificación de la red a velocidades de 10 Gbps.
- **SAI / UPS (APC Smart-UPS 3000VA):** Ubicados en la base del Rack A. Proporcionan autonomía eléctrica y protección contra picos de tensión, asegurando un cierre ordenado de los sistemas ante un corte de suministro prolongado.



1.3 Infraestructura Eléctrica y Cálculo del SAI

El CPD contará con un sistema de alimentación redundante en la Sala Eléctrica. Cada servidor dispone de dos fuentes de alimentación: una conectada a la red principal y otra al Sistema de Alimentación Ininterrumpida (SAI/UPS).

Cálculo matemático del SAI:

- **Consumo total estimado:** Cada servidor Dell consume a pleno rendimiento unos 500W (500W x 4 = 2000W). Los switches y accesorios consumen unos 200W. Consumo total máximo = 2200 Watts.
- **Tiempo de autonomía deseado:** Queremos que los servidores se mantengan encendidos 30 minutos ante un corte de luz para un apagado controlado o esperar al grupo electrógeno.
- **Energía necesaria:** 2200W * 0,5 horas = 1100 Wh.

- **Elección del equipo:** Hemos seleccionado el SAI APC Smart-UPS SRT 3000VA (2700W), que soporta los 2200W de carga.
- **Cálculo de baterías:** Las baterías internas proporcionan 15 minutos a 2200W. Para llegar a los 30 minutos requeridos, añadiremos 1 módulo de baterías externo adicional (Battery Pack), garantizando el tiempo operativo sin corriente de red.

1.4 Seguridad Física y Lógica

Seguridad Física (Ilustrada en el diagrama general)

- **Control de acceso:** Lectores de tarjetas/biométricos en las puertas y registro de logs de entrada y salida.
- **Videovigilancia:** Cámaras de circuito cerrado (CCTV) enfocando los pasillos de los racks y accesos, con retención de grabaciones.
- **Prevención y Extinción de Incendios:** Sensores de humo fotoeléctricos (Detectores de humo). Sistema de extinción automática mediante gas (Fire Suppression Gas System) que ahoga el fuego sin dejar residuos.
- **Vías de evacuación:** Letreros de salida de emergencia e iluminación fotoluminiscente indicando las vías seguras.

Seguridad Lógica

- **Firewalls:** Appliance físico perimetral y firewall por software (ufw/iptables) en cada servidor para bloquear puertos no utilizados.
- **Monitorización:** Implementación de Zabbix para monitorizar en tiempo real el consumo de CPU, RAM, disco y red, con alertas.
- **Copias de seguridad (Backups):** Política 3-2-1. Respaldos incrementales diarios y completos semanales, guardando copias cifradas en un servidor remoto.
- **RAID:** Discos en RAID 10 para las Bases de Datos (velocidad y tolerancia a fallos) y RAID 5 para el almacenamiento de vídeo.

1.5 Prevención de Riesgos Laborales (PRL) en el CPD

Las medidas de prevención de riesgos aplicadas a los técnicos son:

- **Ruido:** Uso obligatorio de protección auditiva si la intervención dura más de 15 minutos (> 80 dB).
- **Cargas pesadas:** Uso de elevadores mecánicos de servidores. Prohibido manipular equipos de más de 15 kg por una sola persona.
- **Riesgo eléctrico:** Uso de pulseras antiestáticas y suelo técnico con recubrimiento disipativo.
- **Climatización:** Uso de ropa de abrigo ligera debido a las bajas temperaturas de los pasillos fríos.

1.6 Implementación del CPD en la nube (AWS)

Como parte integral del proyecto, hemos desplegado una extensión de nuestra infraestructura en Amazon Web Services (AWS) utilizando **3 instancias EC2** con el sistema operativo **Ubuntu Server 24.04 LTS**. No se han utilizado AMIs preconfiguradas, realizando toda la instalación de servicios desde cero.

1.6.1 Guía de Despliegue en la Consola de AWS

Para crear la infraestructura, hemos seguido estos pasos en AWS:

1. **Instancias:** Despliegue de 3 máquinas EC2 **t2.micro**.
2. **Nombres:** Servidor-Web, Servidor-LDAP, Servidor-Logs.
3. **Key Pair:** Utilización de la clave preexistente del entorno de laboratorio llamada **labsuser.pem** para el acceso seguro.
4. **Network Settings (Security Group):** Se han abierto los siguientes puertos:
 - **SSH (22):** Para administración remota.
 - **HTTP/HTTPS (80/443):** Para el acceso al servidor Web.
 - **Custom UDP/TCP (514):** Para el tráfico interno de logs.
 - **All ICMP - IPv4 (Ping):** Para permitir comprobación de conectividad entre las máquinas de la misma VPC.

1.6.2 Configuración Base y Gestión de Accesos

Aplicando el principio de mínimo privilegio, nos conectamos inicialmente a cada servidor utilizando nuestra clave del laboratorio (**ssh -i labsuser.pem ubuntu@IP_PUBLICA**) y realizamos la siguiente fortificación:

1. **Creación de usuario personalizado:** Estamos clonando el permiso de acceso que AWS nos dio originalmente para el usuario **ubuntu** y entregándoselo a nuestro nuevo usuario **innovate_admin**, para que este pueda ser el único administrador real de la máquina.

```
sudo adduser innovate_admin (password: pirineus)
sudo usermod -aG sudo innovate_admin
```

2. **Traspaso de Claves para el nuevo usuario:**

```
sudo mkdir -p /home/innovate_admin/.ssh
sudo cp /home/ubuntu/.ssh/authorized_keys /home/innovate_admin/.ssh/
sudo chown -R innovate_admin:innovate_admin /home/innovate_admin/.ssh
sudo chmod 700 /home/innovate_admin/.ssh
sudo chmod 600 /home/innovate_admin/.ssh/authorized_keys
```

*Terminal entrando con **ssh -i "labsuser.pem" innovate_admin@IP_PÚBLICA o DNS Público**).*

```

cris...@HA208-42-ZLV-058:~/Documents$ ssh -i "labsuser.pem" innovate_
admin@ec2-54-175-218-243.compute-1.amazonaws.com
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1012-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May 18 10:18:42 UTC 2026

System load:  0.0               Processes:            109
Usage of /:   27.3% of 6.71GB   Users logged in:     0
Memory usage: 23%              IPv4 address for enX0: 172.31.43.230
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

```

```

innovate_admin@ip-172-31-43-230:~$ pwd
/home/innovate_admin
innovate_admin@ip-172-31-43-230:~$ ls -la
total 28
drwxr-x--- 4 innovate_admin innovate_admin 4096 May 18 10:18 .
drwxr-xr-x 4 root            root          4096 May 18 10:05 ..
-rw-r--r-- 1 innovate_admin innovate_admin  220 May 18 10:05 .bash_logout
-rw-r--r-- 1 innovate_admin innovate_admin 3771 May 18 10:05 .bashrc
drwx----- 2 innovate_admin innovate_admin 4096 May 18 10:18 .cache
-rw-r--r-- 1 innovate_admin innovate_admin  807 May 18 10:05 .profile
drwxr-xr-x 2 innovate_admin innovate_admin 4096 May 18 10:15 .ssh

```

1.6.3 Direccionamiento de Red y Conectividad

Para que los servidores se comuniquen entre sí (ej. enviar logs o validar SFTP contra el LDAP), hemos definido el siguiente esquema de red interno:

Nombre Servidor	Rol del Servicio	IP Privada (Interna)	IP Pública (Acceso)
Servidor 1 (Cristian)	Web + SFTP	172.31.20.252	3.230.189.112
Servidor 2 (Marc)	Directorio Activo (LDAP)	172.31.25.65	44.206.30.106
Servidor 3	Logs Centralizados	172.31.26.253	54.175.41.208

(Jesus)			
Servidor 4 (Jesus)	Base de datos (MariaDB)	172.31.22.189	54.146.49.143

Comprobación de Conectividad (Ping): Gracias a la regla ICMP, hemos verificado que las máquinas se ven entre sí usando sus IPs públicas.

Ping del servidor 2 y 3 al servidor 1:

Server:2

```
innovate_admin@ip-172-31-25-65:~$ ping 3.230.189.112

PING 3.230.189.112 (3.230.189.112) 56(84) bytes of data.
64 bytes from 3.230.189.112: icmp_seq=1 ttl=63 time=0.650 ms
64 bytes from 3.230.189.112: icmp_seq=2 ttl=63 time=0.597 ms
64 bytes from 3.230.189.112: icmp_seq=3 ttl=63 time=0.638 ms
^C
--- 3.230.189.112 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.597/0.628/0.650/0.022 ms
innovate_admin@ip-172-31-25-65:~$
```

Server 3:

```
innovate_admin@ip-172-31-26-253:~$ ping 3.230.189.112

PING 3.230.189.112 (3.230.189.112) 56(84) bytes of data.
64 bytes from 3.230.189.112: icmp_seq=1 ttl=63 time=0.672 ms
64 bytes from 3.230.189.112: icmp_seq=2 ttl=63 time=1.48 ms
64 bytes from 3.230.189.112: icmp_seq=3 ttl=63 time=0.941 ms
^C
```

1.6.4 Distribución y Rol de los Servicios

Para optimizar el rendimiento y la seguridad, hemos distribuido las funciones de la infraestructura en tres servidores independientes, evitando así puntos únicos de fallo y conflictos de recursos:

- **Servidor 1 (Web & SFTP):** Actúa como la cara pública de Innovate Tech. Aloja el servidor web **Nginx** para la plataforma corporativa y un servicio **SFTP (OpenSSH)** configurado con Chroot Jail. La autenticación de este servicio se delega al servidor LDAP para mantener una gestión de usuarios centralizada.
 - **Servidor 2 (LDAP):** Este nodo ejecuta el servicio **Slapd**. Es el corazón de la seguridad lógica, almacenando de forma centralizada las cuentas de usuario, grupos y permisos que utilizan los demás servidores para validar accesos.
 - **Servidor 3 (Logs Centralizados):** Ejecuta **Rsyslog** configurado en modo servidor. Su función es recolectar y almacenar todos los eventos de sistema y seguridad de los Servidores 1 y 2, facilitando la auditoría y la detección de errores desde un único punto.
-

1.6.5 Automatización con Ansible

Siguiendo las mejores prácticas de infraestructura como Código (IaC), hemos automatizado el despliegue de los Servidores 1 y 3. Para ello, hemos utilizado una estación de control local para ejecutar los siguientes archivos de configuración:

Fichero: `inventory`

Este archivo define la topología de red y las credenciales de acceso para la automatización. Ini, TOML

```
[web_servers]
```

```
servidor_web ansible_host=IP_PUBLICA_S1
```

```
[log_servers]
```

```
servidor_logs ansible_host=IP_PUBLICA_S3
```

```
[all:vars]
```

```
ansible_user=innovate_admin
```

```
ansible_ssh_private_key_file=labsuser.pem
```

```
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

```
GNU nano 6.2                                inventory
[web_servers]
servidor1 ansible_host=3.85.82.239

[log_servers]
servidor3 ansible_host=18.208.225.149 ansible_ssh_private_key_file=/home/cristian.ojeda.7e7/Documents/llave_socio.pem

[all:vars]
ansible_user=innovate_admin
ansible_ssh_private_key_file=/home/cristian.ojeda.7e7/Documents/labsuser.pem
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Fichero: `playbook.yml`

El libro de estrategias que ejecuta la instalación desatendida y la personalización de los servicios.

YAML

- name: Automatización del Nodo Web (Servidor 1)

hosts: web_servers

become: yes

tasks:

- name: Actualizar repositorios e instalar Nginx

apt:

name: nginx

state: present

update_cache: yes

- name: Personalizar página de inicio corporativa

copy:

content: "<h1>Bienvenido a la plataforma de Innovate Tech</h1>"

dest: /var/www/html/index.html

owner: www-data

group: www-data

- name: Asegurar que Nginx esté iniciado y habilitado

 - service:

 - name: nginx

 - state: started

 - enabled: yes

- name: Automatización del Nodo de Logs (Servidor 3)

 - hosts: log_servers

 - become: yes

 - tasks:

 - name: Instalar y asegurar servicio Rsyslog

 - apt:

 - name: rsyslog

 - state: present

- name: Habilitar el servicio para arranque automático

 - service:

 - name: rsyslog

 - state: started

 - enabled: yes

```
GNU nano 6.2                                playbook.yml
---
- name: Automatización del Nodo Web (Servidor 1)
  hosts: web_servers
  become: yes
  tasks:
    - name: Actualizar repositorios e instalar Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Personalizar página de inicio corporativa
      copy:
        content: "<h1>Bienvenido a la plataforma de Innovate Tech</h1>"
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data

    - name: Asegurar que Nginx esté iniciado y habilitado
      service:
        name: nginx
        state: started
        enabled: yes

- name: Automatización del Nodo de Logs (Servidor 3)
  hosts: log_servers
  become: yes
  tasks:
    - name: Instalar y asegurar servicio Rsyslog
      apt:
        name: rsyslog
        state: present

    - name: Habilitar el servicio para arranque automático
      service:
        name: rsyslog
        state: started
        enabled: yes
```

Ejecución y Verificación

La implementación se realiza mediante el comando **ansible-playbook -i inventory playbook.yml**. Este proceso garantiza que la configuración sea idéntica y libre de errores humanos en ambos servidores.

```

cristian.ojeda.7e7@HA208-42-ZLV-058:~/Documents$ ansible-playbook -i inventory playbook.yml -K
BECOME password:

PLAY [Automatización del Nodo Web (Servidor 1)] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host servidor1 is using the discovered Python interpreter at
/usr/bin/python3.12, but future installation of another Python interpreter could change the meaning
of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [servidor1]

TASK [Actualizar repositorios e instalar Nginx] *****
ok: [servidor1]

TASK [Personalizar página de inicio corporativa] *****
ok: [servidor1]

TASK [Asegurar que Nginx esté iniciado y habilitado] *****
ok: [servidor1]

PLAY [Automatización del Nodo de Logs (Servidor 3)] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host servidor3 is using the discovered Python interpreter at
/usr/bin/python3.12, but future installation of another Python interpreter could change the meaning
of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [servidor3]

TASK [Instalar y asegurar servicio Rsyslog] *****
changed: [servidor3]

TASK [Habilitar el servicio para arranque automático] *****
ok: [servidor3]

PLAY RECAP *****
servidor1      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
servidor3      : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

Comprobaciones:

Servidor 1:

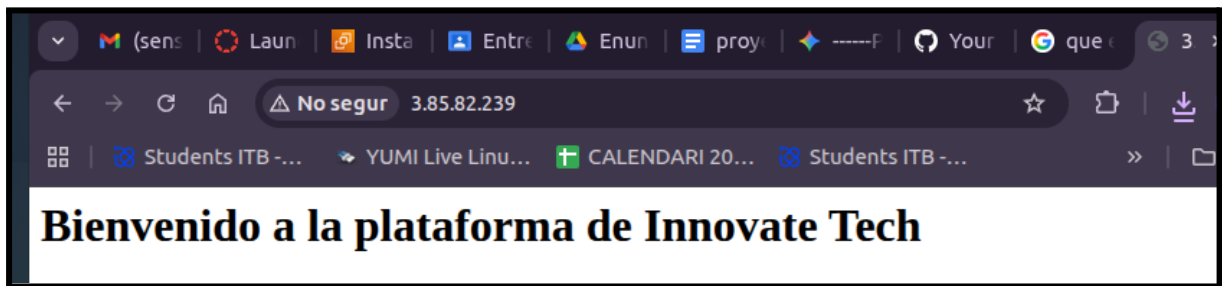
Servidor 3:

```

innovate_admin@ip-172-31-26-253:~$ sudo systemctl status rsyslog
[sudo] password for innovate_admin:
● rsyslog.service - System Logging Service
   Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-05-19 07:48:12 UTC; 1min 0s ago
   TriggeredBy: ● syslog.socket
     Docs: man:rsyslogd(8)
           man:rsyslog.conf(5)
           https://www.rsyslog.com/doc/
   Process: 4086 ExecStartPre=/usr/lib/rsyslog/reload-apparmor-profile (code=exited, status=0/SUCCESS)
   Main PID: 4091 (rsyslogd)
     Tasks: 4 (limit: 1117)
    Memory: 1.3M (peak: 4.9M)
       CPU: 130ms
    CGroup: /system.slice/rsyslog.service
            └─4091 /usr/sbin/rsyslogd -n -iNONE

```

Entrar a la página web desde cualquier navegador:



1. Servidor 2: Directorio Activo (LDAP) - Configuración Manual

Este es el primer servidor que debe estar listo, ya que los demás dependen de él para validar usuarios.

Paso 1: Instalación

```
sudo apt update
```

```
sudo apt install slapd ldap-utils -y
```

```
innovate_admin@ip-172-31-25-40:~$ sudo apt install slapd ldap-utils -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libltdl7 libodbc2
```

contraseña: *pirineus*

Paso 2: Configurar el dominio Si no te pidió el dominio durante la instalación, ejecuta:

```
sudo dpkg-reconfigure slapd
```

- ¿Omitir la configuración? **No**
- Nombre de dominio: **innovate.tech**
- Nombre de la organización: **Innovate**

Paso 3: Crear la estructura (El "guion" LDIF) Crea un archivo llamado **estructura.ldif**:

LDIF

Creación de la Unidad Organizativa para Usuarios

dn: ou=People,dc=innovate,dc=cat

objectClass: organizationalUnit

ou: People

Creación de un usuario de prueba

dn: cn=innovate_user1,ou=People,dc=innovate,dc=cat

objectClass: inetOrgPerson

objectClass: posixAccount

objectClass: shadowAccount

cn: innovate_user1

sn: User1

uid: innovate_user1

uidNumber: 10001

gidNumber: 10001

homeDirectory: /home/innovate_user1

loginShell: /bin/bash

userPassword: password

```
GNU nano 7.2 estructura.ldif
# Creación de la Unidad Organizativa para Usuarios
dn: ou=People,dc=innovate,dc=cat
objectClass: organizationalUnit
ou: People

# Creación de un usuario de prueba
dn: cn=innovate_user1,ou=People,dc=innovate,dc=cat
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: innovate_user1
sn: User1
uid: innovate_user1
uidNumber: 10001
gidNumber: 10001
homeDirectory: /home/innovate_user1
loginShell: /bin/bash
userPassword: pirineus
```

Paso 4: Cargar la estructura al servidor

`ldapadd -x -D "cn=admin,dc=innovate,dc=tech" -W -f estructura.ldif`

```
creating LDAP directory... done.
innovate_admin@ip-172-31-25-40:~$ ldapadd -x -D "cn=admin,dc=innovate,dc=cat" -W -f estructura.ldif
Enter LDAP Password:
adding new entry "ou=People,dc=innovate,dc=cat"

adding new entry "cn=innovate_user1,ou=People,dc=innovate,dc=cat"
innovate_admin@ip-172-31-25-40:~$
```

2. Servidor 3: Logs Centralizados (Recolector)

Este servidor recibirá la información de los otros dos.

Paso 1: Configurar la escucha (Receiver) Edita el archivo de configuración:

`sudo nano /etc/rsyslog.conf`

Busca y **descomenta** (quita el #) estas 4 líneas:

`module(load="imudp")`

`input(type="imudp" port="514")`

`module(load="imtcp")`

`input(type="imtcp" port="514")`

```

GNU nano 7.2 /etc/rsyslog.conf
# /etc/rsyslog.conf configuration file for rsyslog
#
# For more information install rsyslog-doc and see
# /usr/share/doc/rsyslog-doc/html/configuration/index.html
#
# Default logging rules can be found in /etc/rsyslog.d/50-default.conf

#####
#### MODULES ####
#####

module(load="imuxsock") # provides support for local system logging
#module(load="immark") # provides --MARK-- message capability

# provides UDP syslog reception
module(load="imudp")
input(type="imudp" port="514")

# provides TCP syslog reception
module(load="imtcp")
input(type="imtcp" port="514")

# provides kernel logging support and enable non-kernel klog messages
module(load="imklog" permitnonkernelfacility="on")

#####
#### GLOBAL DIRECTIVES ####
#####

# Filter duplicated messages
$RepeatedMsgReduction on

```

Paso 2: Reiniciar y verificar

sudo systemctl restart rsyslog

Comprobar que escucha en el puerto 514

ss -tunlp | grep 514

```

innovate_admin@ip-172-31-26-253:~$ ss -tunlp | grep 514
udp    UNCONN 0      0      0.0.0.0:514      0.0.0.0:*
udp    UNCONN 0      0      [::]:514        [::]:*
tcp    LISTEN 0      25     0.0.0.0:514      0.0.0.0:*
tcp    LISTEN 0      25     [::]:514        [::]:*
innovate_admin@ip-172-31-26-253:~$ 

```

3. Servidor 1: Web + SFTP (Emisor de Logs y Cliente LDAP)

Este servidor es el más complejo porque hace tres cosas a la vez.

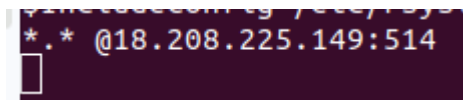
A. Envío de Logs al Servidor 3

Editar el archivo de rsyslog:

```
sudo nano /etc/rsyslog.conf
```

Añade esta línea al final (sustituir la IP por la **IP PÚBLICA** del Servidor 3):

```
*.* @54.159.250.103:514
```



(Usar @ para UDP, es más rápido para logs).

Reiniciar rsyslog: `sudo systemctl restart rsyslog`

B. Unión con el LDAP (Para que el SFTP reconozca a los usuarios)

Debemos instalar el cliente LDAP para que el sistema "sepa" que existe el usuario **innovate_user1** del servidor LDAP:

```
sudo apt update && sudo apt install libnss-ldap libpam-ldap nslcd ldap-utils -y
```

- URI de LDAP: **ldap://IP_PÚBLICA_SERVIDOR_2**
- DN de la base: **dc=innovate,dc=cat**

Editar `sudo nano /etc/nsswitch.conf`:

```
passwd:      files systemd ldap
```

```
group:       files systemd ldap
```

```
shadow:      files
```

Activar la **contraseña** de **LDAP**: Ejecutar **sudo pam-auth-update** y marca con un asterisco [*] LDAP Authentication

C. Preparar la "Cárcel" (Prevención de bloqueos):

Antes de tocar el SSH, debemos crear la carpeta del **usuario** y hacer que el dueño sea **root**. Si no hacemos esto, SSH nos cerrará la conexión.

```
sudo groupadd -g 10005 innovate_users
```

```
sudo mkdir -p /home/innovate_user1
```

```
sudo chown root:root /home/innovate_user1
```

```
sudo chmod 755 /home/innovate_user1
```

D. Configuración de SFTP con Chroot (Seguridad)

Ahora que el usuario existe (por LDAP) y su carpeta está creada, ya puedo editar el SSH.

Ejecutar: `sudo nano /etc/ssh/sshd_config`

Añadir esto al final del archivo para que los usuarios no puedan "cotillear" otras carpetas:

Regla estricta para usuarios normales de LDAP (solo SFTP)

Match Group innovate_users

ChrootDirectory /home/%u

ForceCommand internal-sftp

PasswordAuthentication yes

AllowTcpForwarding no

X11Forwarding no

Regla explícita para mi usuario administrador (consola total + tu llave .pem)

Match User innovate_admin

PasswordAuthentication no

PubkeyAuthentication yes

Comprobar que no hay errores: `sudo sshd -t`

Si no hay errores, **reiniciar**: `sudo systemctl restart ssh`

Máquina	Herramienta	Acción Clave
S1 (Web)	Ansible	Instalar Nginx y la página index.html.
S1 (Web)	SSH/PAM	Configurar Chroot y conectar con S2 para el login del SFTP.
S2 (LDAP)	Slapd	Crear el árbol dc=innovate,dc=cat y el usuario innovate_user1.
S3 (Logs)	Rsyslog	Activar módulos imudp e imtcp para recibir datos de S1 y S2.

```

innovate_admin@ip-172-31-25-40:~$ ldapsearch -x -b "dc=innovate,dc=cat" "(uid=innovate_user1)"
# extended LDIF
#
# LDAPv3
# base <dc=innovate,dc=cat> with scope subtree
# filter: (uid=innovate_user1)
# requesting: ALL
#
# innovate_user1, People, innovate.cat
dn: cn=innovate_user1,ou=People,dc=innovate,dc=cat
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: innovate_user1
sn: User1
uid: innovate_user1
uidNumber: 10001
gidNumber: 10001
homeDirectory: /home/innovate_user1
loginShell: /bin/bash

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1

```

Aquí podemos ver como vemos el usuario y también con el getent:

```

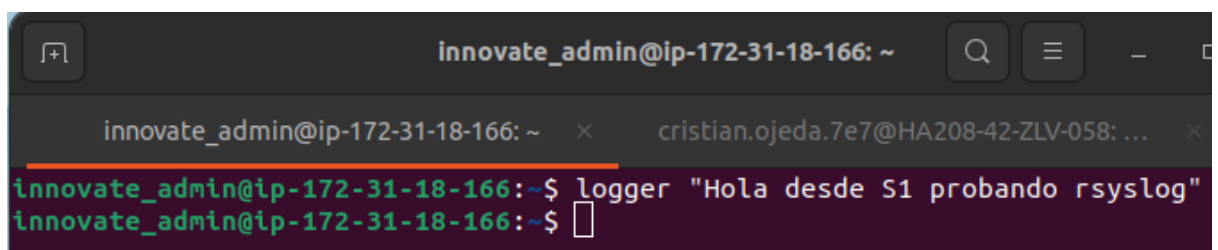
innovate_admin@ip-172-31-18-166:~$ getent passwd innovate_user1
innovate_user1:x:10001:10001:innovate_user1:/home/innovate_user1:/bin/bash
innovate_admin@ip-172-31-18-166:~$

```

Cómo probarlo todo (Para las capturas finales):

1. Desde S1 o S2: Ejecutar **logger "Hola desde el nodo X"**.
2. En S3: Ejecutar **tail -f /var/log/syslog**. Debemos ver el mensaje de los otros servidores.

S1:

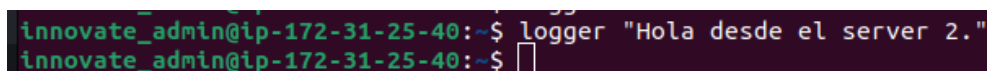


```

innovate_admin@ip-172-31-18-166: ~
innovate_admin@ip-172-31-18-166: ~ x cristian.ojeda.7e7@HA208-42-ZLV-058: ... x
innovate_admin@ip-172-31-18-166:~$ logger "Hola desde S1 probando rsyslog"
innovate_admin@ip-172-31-18-166:~$

```

S2:



```

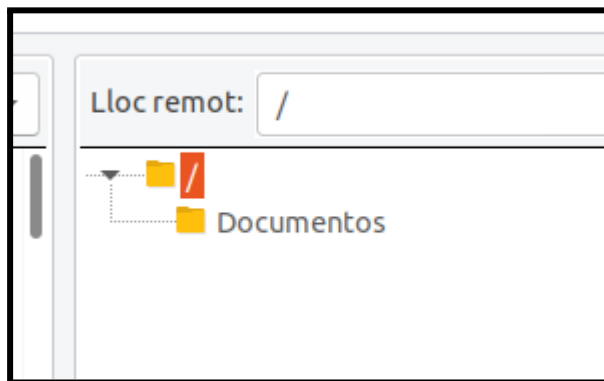
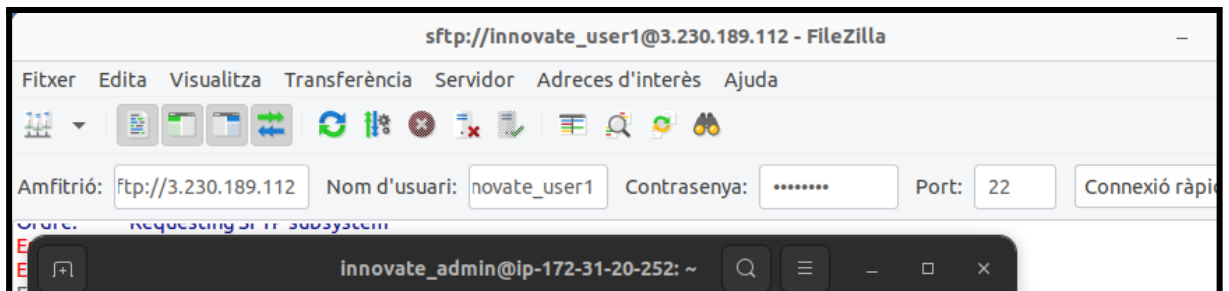
innovate_admin@ip-172-31-25-40:~$ logger "Hola desde el server 2."
innovate_admin@ip-172-31-25-40:~$

```

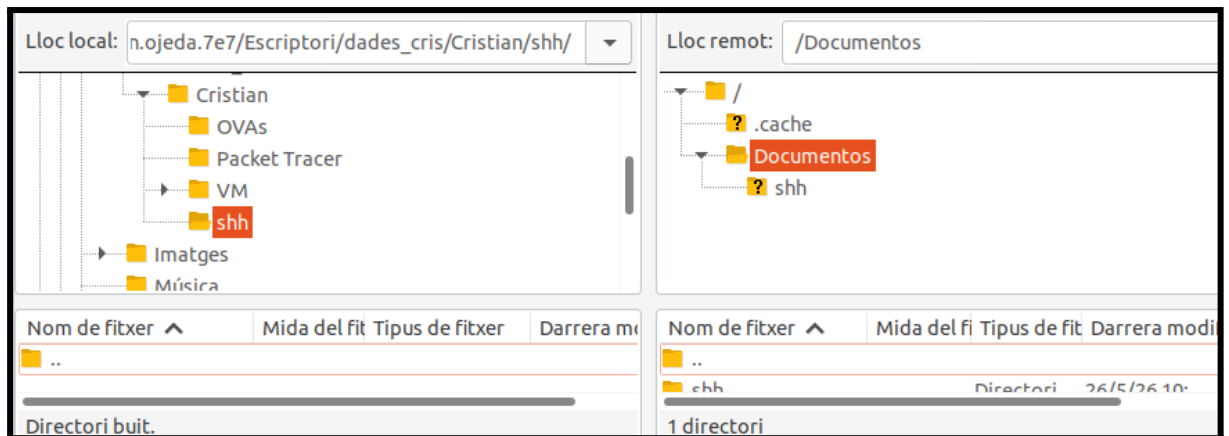
S3: Comprobación **tail -f /var/log/syslog**

```
SERVER 2j
2026-05-20T10:21:36+00:00 ip-172-31-25-40 innovate_admin: Hola desde el server 2.
2026-05-20T10:21:39+00:00 ip-172-31-18-166 innovate_admin: Hola desde S1 probando rsyslog
```

3. **Desde el PC:** Intentar entrar por SFTP (**FileZilla**) con el usuario **innovate_user1** y la contraseña **pirineus** (**puerto 22**). Si entramos, la integración de S1-S2 es perfecta.



```
innovate_admin@ip-172-31-18-166: ~ × cristian.ojeda.7e7@HA208-42-ZL
innovate_admin@ip-172-31-18-166:~$ ls -l /home/innovate_user1/
total 4
drwxr-xr-x 2 root root 4096 May 20 09:48 Documentos
innovate_admin@ip-172-31-18-166:~$
```



```
innovate_admin@ip-172-31-20-252:~$ sudo ls -l /home/innovate_user1/Documentos
total 4
drwxr-xr-x 2 innovate_user1 innovate_users 4096 May 26 08:07 ssh
```

Aquí podemos confirmar que está bien hecho. Al poder conectarnos demuestra que tenemos conexión con el Servidor 2 LDAP, porque yo el Servidor 1 no he creado el usuario innovate_user1

2 Implantación de los servicios de audio y vídeo

2.1 Descripción general

Como responsable de la infraestructura multimedia de Innovate Tech, he decidido centralizar todos los servicios de streaming y comunicación a nuestro Servidor 1 (IP Pública: **3.230.189.112**). Dado que este nodo ya actúa como servidor web principal corporativo, aprovecharé esta misma máquina para albergar los servicios de streaming de vídeo y audio, además de implementar la plataforma de videoconferencias interna.

Toda la solución está orientada a la nube (AWS), y diseñada para funcionar con protocolos estándar abiertos, permitiendo a los empleados el acceso desde clientes locales o mediante navegadores web sin necesidad de instalar software alguno de terceros.

2.2 Implantación del servicio de audio

Para el servicio de distribución de audio (radio interna o comunicados de voz), he implementado Icecast2, un servidor de streaming robusto y ligero que permite emitir contenidos de audio digital bajo demanda o en directo.

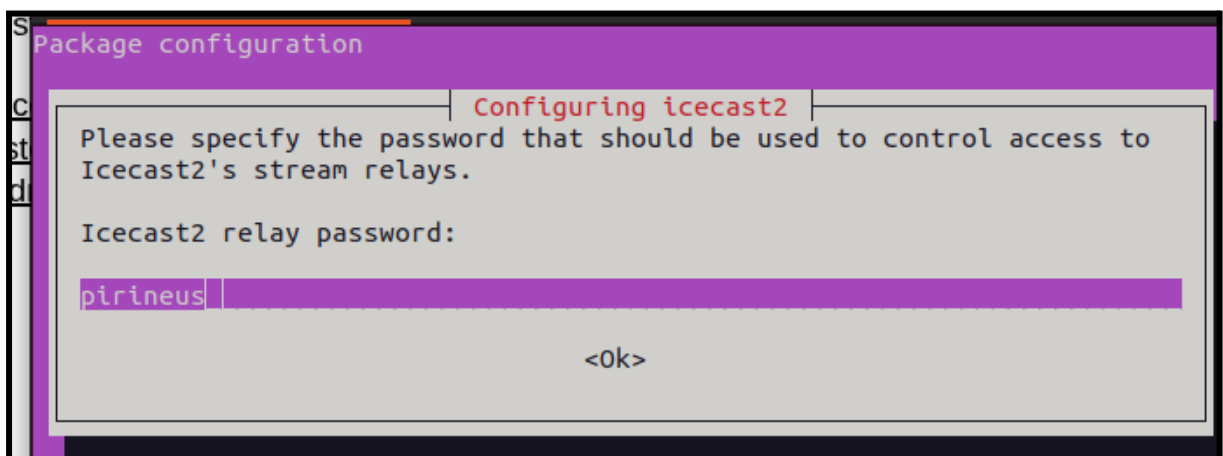
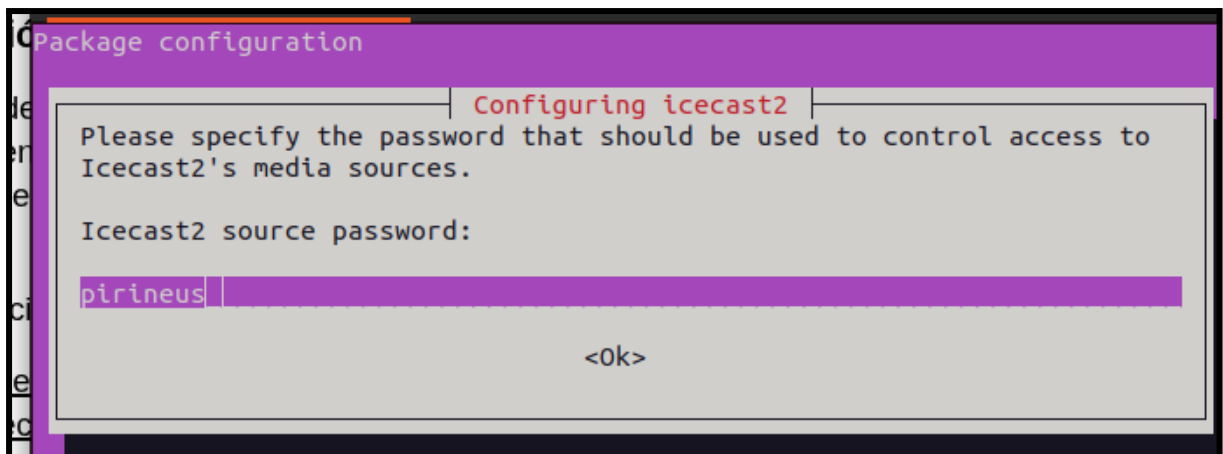
Pasos de instalación y configuración (comandos)

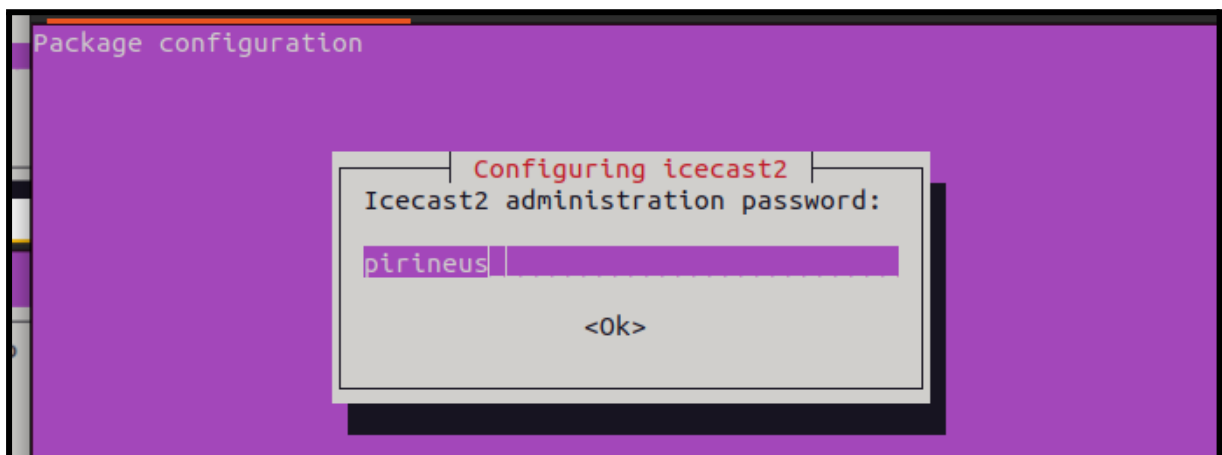
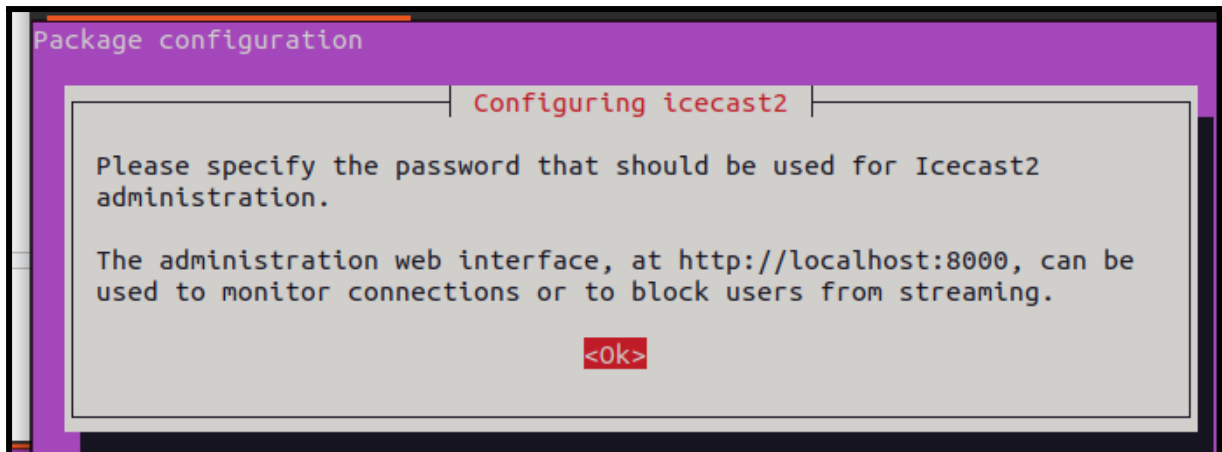
- 1. Primero, me he conectado por SSH al Servidor 1 y he instalado el paquete necesario:**

```
sudo apt update
```

```
sudo apt install icecast2 -y
```

- 2. Durante el proceso de instalación, el asistente pide el nombre de dominio (he puesto localhost) y las contraseñas para los usuarios source, relay y admin (he utilizado la contraseña estándar del proyecto: pirineus).**





3. Una vez finalizada la instalación, he habilitado y arrancado el servicio:

```
sudo systemctl enable icecast2
```

```
sudo systemctl start icecast2
```

```
sudo systemctl status icecast2
```

```
innovate_admin@ip-172-31-20-252:~$ sudo systemctl enable icecast2
icecast2.service is not a native service, redirecting to systemd-sysv-install
.
Executing: /usr/lib/systemd/systemd-sysv-install enable icecast2
innovate_admin@ip-172-31-20-252:~$ sudo systemctl start icecast2
innovate_admin@ip-172-31-20-252:~$ sudo systemctl status icecast2
● icecast2.service - LSB: Icecast2 streaming media server
   Loaded: loaded (/etc/init.d/icecast2; generated)
   Active: active (running) since Tue 2026-05-26 08:22:25 UTC; 1min 36s ago
     Docs: man:systemd-sysv-generator(8)
    Tasks: 4 (limit: 1117)
   Memory: 4.9M (peak: 5.4M)
      CPU: 65ms
   CGroup: /system.slice/icecast2.service
           └─5118 /usr/bin/icecast2 -b -c /etc/icecast2/icecast.xml

May 26 08:22:25 ip-172-31-20-252 systemd[1]: Starting icecast2.service - LSB>
May 26 08:22:25 ip-172-31-20-252 icecast2[5090]: * Starting streaming media>
May 26 08:22:25 ip-172-31-20-252 icecast2[5090]: ...done.
May 26 08:22:25 ip-172-31-20-252 systemd[1]: Started icecast2.service - LSB:>
May 26 08:22:25 ip-172-31-20-252 icecast2[5118]: [2026-05-26 08:22:25] WARN>
lines 1-15...skipping...
icecast2.service - LSB: Icecast2 streaming media server
```

4. Integración web: Para permitir el acceso vía navegador web de forma transparente, he integrado la etiqueta HTML5 en la página corporativa (</var/www/html/index.html>). Este código apunta al punto de montaje de nuestra radio en formato .ogg:

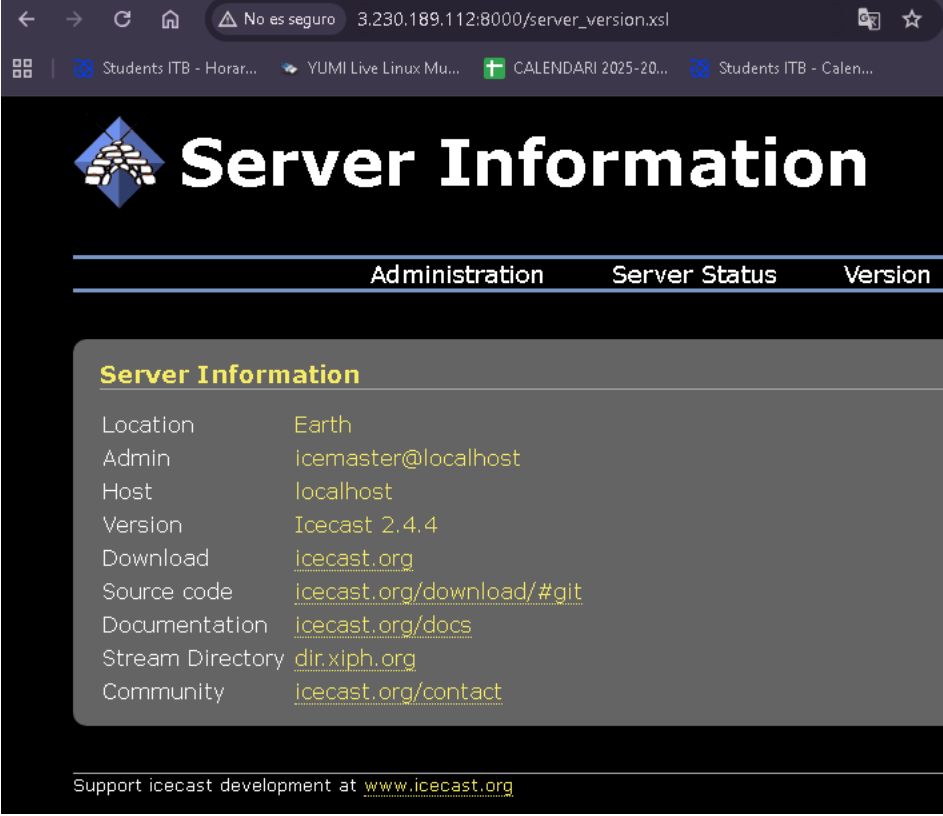
```
<audio controls preload="none">
  <source src="http://3.230.189.112:8000/radio_interna.ogg" type="audio/ogg">
  <source src="http://3.230.189.112:8000/radio_interna.mp3" type="audio/mpeg">
  El teu navegador no suporta el format d'àudio digital.
</audio>
```

5. Configuración y Estado del Servidor de Streaming (Iccast2)

Se ha instalado y configurado exitosamente el servidor Iccast2 en la instancia de AWS para gestionar la transmisión de audio. El servicio se encuentra activo y a la escucha en el puerto 8000.

A continuación, se adjuntan las evidencias de la disponibilidad del servidor y del reproductor web:

1. El servidor Iccast:



The screenshot shows a web browser window displaying the 'Server Information' page of the Iccast2 server. The browser's address bar shows the URL '3.230.189.112:8000/server_version.xml'. The page has a dark theme with a blue and white logo on the left. The main heading is 'Server Information'. Below this, there are three tabs: 'Administration', 'Server Status', and 'Version'. The 'Server Information' tab is active, showing a list of server details:

Server Information	
Location	Earth
Admin	icemaster@localhost
Host	localhost
Version	Iccast 2.4.4
Download	icecast.org
Source code	icecast.org/download/#git
Documentation	icecast.org/docs
Stream Directory	dir.xiph.org
Community	icecast.org/contact

At the bottom of the page, there is a footer that says 'Support icecast development at www.icecast.org'.

2. El servicio en Linux:

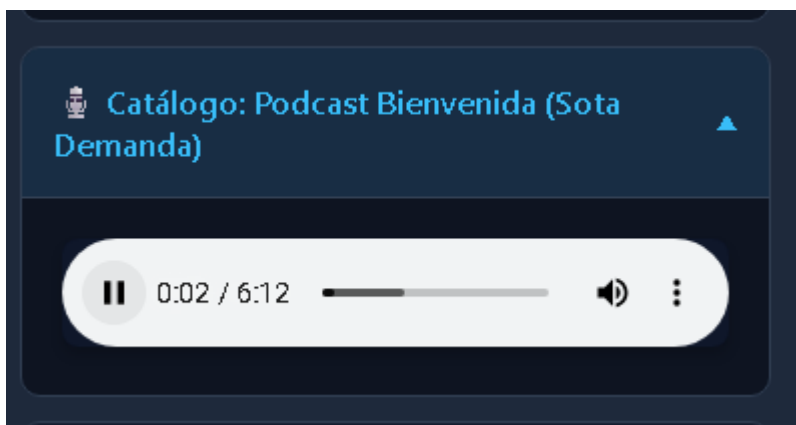
```

innovate_admin@ip-172-31-20-252:~$ sudo systemctl status icecast2
[sudo] password for innovate_admin:
● icecast2.service - LSB: Icecast2 streaming media server
   Loaded: loaded (/etc/init.d/icecast2; generated)
   Active: active (running) since Wed 2026-05-27 17:03:56 UTC; 11min ago
     Docs: man:systemd-sysv-generator(8)
  Process: 506 ExecStart=/etc/init.d/icecast2 start (code=exited, status=0/SUCCESS)
    Tasks: 4 (limit: 1117)
   Memory: 8.8M (peak: 9.0M)
      CPU: 299ms
   CGroup: /system.slice/icecast2.service
           └─704 /usr/bin/icecast2 -b -c /etc/icecast2/icecast.xml

May 27 17:03:54 ip-172-31-20-252 systemd[1]: Starting icecast2.service - LSB: Icecast2 streaming media server:
May 27 17:03:55 ip-172-31-20-252 icecast2[506]: * Starting streaming media server icecast2
May 27 17:03:55 ip-172-31-20-252 icecast2[506]: ...done.
May 27 17:03:55 ip-172-31-20-252 icecast2[704]: [2026-05-27 17:03:55] WARN CONFIG/_parse_root Warning:
May 27 17:03:55 ip-172-31-20-252 icecast2[704]: [2026-05-27 17:03:55] WARN CONFIG/_parse_root Warning:
May 27 17:03:55 ip-172-31-20-252 icecast2[704]: [2026-05-27 17:03:55] WARN CONFIG/_parse_root Warning:
May 27 17:03:56 ip-172-31-20-252 systemd[1]: Started icecast2.service - LSB: Icecast2 streaming media server:

```

3. La web funcionando:



audios:

```

innovate_admin@ip-172-31-20-252:~$ ls -l /var/www/html/
total 24
drwxr-xr-x 2 www-data www-data 4096 May 26 10:53 audios
-rw-r--r-- 1 www-data www-data 12161 May 26 11:50 index.html
-rw-r--r-- 1 root root 615 May 25 10:34 index.nginx-debian.html
drwxr-xr-x 2 www-data www-data 4096 May 26 10:20 videos
innovate_admin@ip-172-31-20-252:~$ ls -l /var/www/html/audios/
total 8772
-rw-r--r-- 1 root root 28915 May 21 09:07 mensaje_ceo.mp3
-rw-r--r-- 1 root root 81 May 26 11:13 playlist.m3u
-rw-r--r-- 1 root root 8945229 Mar 5 2011 podcast_bienvenida.mp3

```

2.2.1 Configuración y Despliegue del Servidor Icecast2

El servidor de streaming se ha desplegado correctamente sobre la instancia EC2. Se ha verificado la apertura del puerto 8000 en los Security Groups de AWS y la correcta ejecución del servicio en segundo plano.

- **Evidencia de Configuración:** En el archivo **icecast.xml** se ha definido el socket de escucha y las credenciales de administración, permitiendo la gestión remota del servidor.
- **Estado del Servicio:** El panel de administración es accesible, lo que valida que el servidor está listo para recibir conexiones de fuentes externas (Source Clients). La ausencia de puntos de montaje activos en la captura adjunta responde a que el emisor se activa bajo demanda para optimizar los recursos de CPU de la instancia.

2.2.2 Servicio de Audio bajo Demanda (Podcasts)

Gestión y Transferencia Segura de Archivos de Audio

Para garantizar la integridad de la biblioteca multimedia y permitir a los administradores realizar copias de seguridad o auditorías de los archivos de audio (.mp3 / .ogg) alojados en el servidor, se utiliza el protocolo SCP (Secure Copy). Este comando permite descargar de forma cifrada y recursiva el contenido desde el entorno de producción en AWS hacia las estaciones de trabajo locales.

- **Comando utilizado desde la máquina local:**

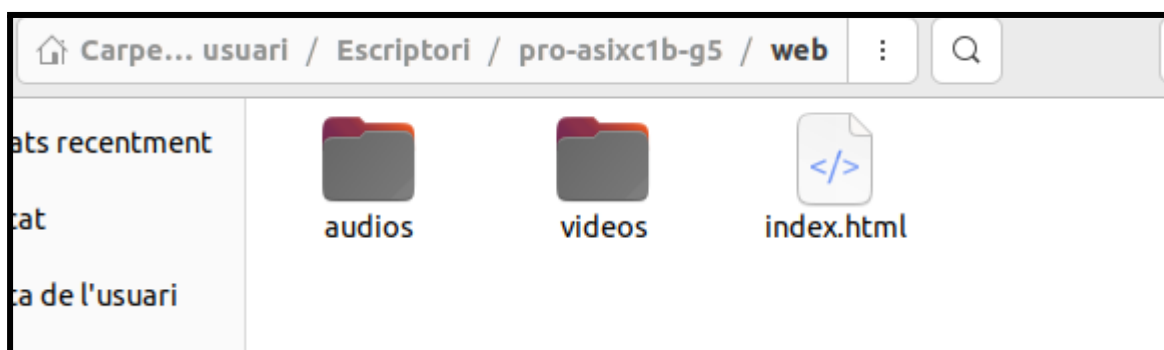
```
scp -r -i labsuser.pem innovate_admin@3.230.189.112:/var/www/html/audios /home/cristian.ojeda.7e7/Escriptori/pro-asixc1b-g5/web/
```

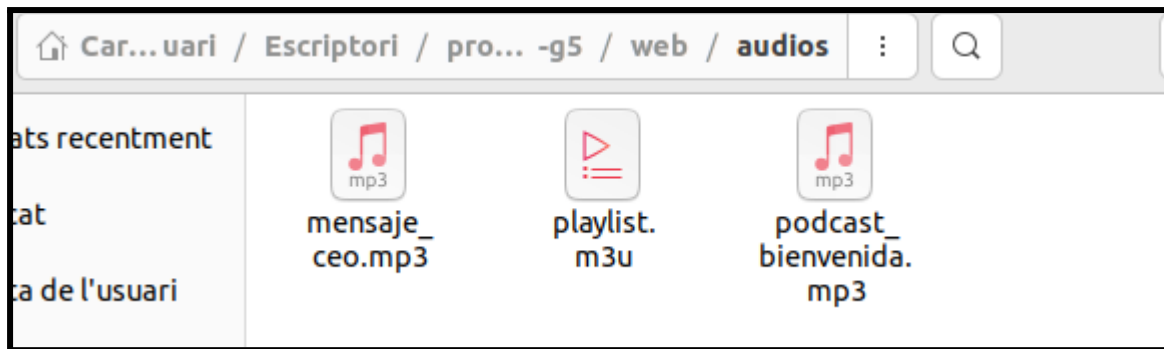
- **Evidencia de transferencia:**

```

cristian.ojeda.7e7@HA208-42-ZLV-058:~/Documents$ scp -r -i labsuser.pem innovate_admin@ec2-3-230-189-112.compute-1.amazonaws.com:/var/www/html/audios /home/cristian.ojeda.7e7/Escriptori/pro-asixc1b-g5/web/
podcast_bienvenida.mp3      100% 8736KB   6.7MB/s   00:01
playlist.m3u                100%   81       0.7KB/s   00:00
mensaje_ceo.mp3             100%  28KB    243.6KB/s  00:00

```





2.3 Implantación del servicio de vídeo

Esta fase se divide en dos partes para cumplir con los requerimientos: streaming de vídeos pregrabados (formación) y el sistema de videoconferencias en directo.

2.3.1 Streaming de Vídeo (VoD)

He aprovechado el servidor Nginx existente instalando el módulo RTMP, que permite servir archivos MP4 de manera muy eficiente.

1. Instalación del módulo RTMP:

```
sudo apt install libnginx-mod-rtmp -y
```

2. Configuración del blog de streaming: He editado el archivo principal de Nginx (sudo nano /etc/nginx/nginx.conf) y he añadido el siguiente blog al final del archivo:

```
rtmp {  
    server {  
        listen 1935;  
        chunk_size 4096;  
  
        application vod {  
            play /var/www/html/videos;  
        }  
    }  
}
```

3. He creado el directorio para los vídeos en formato H.264/MP4, he ajustado los permisos y he reiniciado el servidor web:

```
sudo mkdir -p /var/www/html/videos  
sudo chown -R www-data:www-data /var/www/html/videos  
sudo systemctl restart nginx
```

4. Por último, lo he integrado en la página web de la empresa:

 <video width="640" height="360" controls> <source
src="http://3.230.189.112/videos/video_corporatiu.mp4" type="video/mp4"> El
teu navegador no suporta l'etiqueta de vídeo. </video>

ruta en la instantánea:

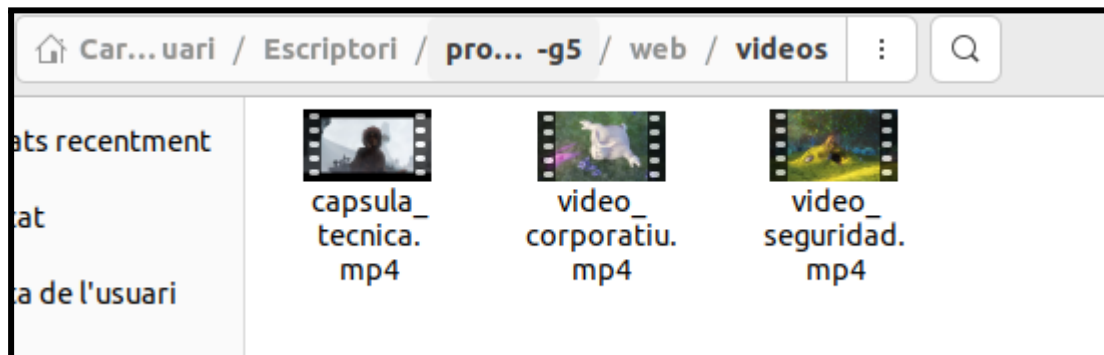
```
innovate_admin@ip-172-31-20-252:~$ ls -l /var/www/html/videos/  
total 2744  
-rwxr-xr-x 1 www-data www-data 1047954 Nov 26 2021 capsula_tecnica.mp4  
-rwxr-xr-x 1 www-data www-data 788493 May 21 09:07 video_corporatiu.mp4  
-rwxr-xr-x 1 www-data www-data 969201 Mar 22 2019 video_seguridad.mp4  
innovate_admin@ip-172-31-20-252:~$
```

Despliegue del Contenido de la Videoteca Corporativa

Al igual que con el servicio de audio, los módulos formativos en formato MP4 almacenados en el servidor Nginx se descargan hacia el repositorio local del grupo de trabajo mediante el uso de SCP recursivo. Esto asegura que el equipo de administración disponga de una copia exacta de los recursos multimedia para su posterior empaquetado y entrega en el repositorio de control de versiones (GitHub).

- **Comando de sincronización:**

```
cristian.ojeda.7e7@HA208-42-ZLV-058:~/Documents$ scp -r -i labsuser.pem innovate  
_admin@ec2-3-230-189-112.compute-1.amazonaws.com:/var/www/html/videos /home/cris  
tian.ojeda.7e7/Escriptori/pro-asixc1b-g5/web/  
capsula_tecnica.mp4 100% 1023KB 1.3MB/s 00:00  
video_seguridad.mp4 100% 946KB 7.4MB/s 00:00  
video_corporatiu.mp4 100% 770KB 6.1MB/s 00:00  
cristian.ojeda.7e7@HA208-42-ZLV-058:~/Documents$
```



2.3.2 Sistema de Videoconferencia (WebRTC)

Para cumplir con el requerimiento de implantar una herramienta de videoconferencia en directo utilizando **WebRTC**, he optado por la instalación de **Jitsi Meet** en el mismo servidor. Este protocolo permite comunicación *peer-to-peer* (P2P) de baja latencia, ideal para reuniones de departamento.

1. He añadido el repositorio oficial de Jitsi y sus claves GPG:

```
curl https://download.jitsi.org/jitsi-key.gpg.key | sudo sh -c 'gpg --dearmor >
/usr/share/keyrings/jitsi-keyring.gpg' echo 'deb
[signed-by=/usr/share/keyrings/jitsi-keyring.gpg] https://download.jitsi.org
stable/' | sudo tee /etc/apt/sources.list.d/jitsi-stable.list
```

2. He realizado la instalación:

```
sudo apt update
sudo apt install jitsi-meet -y
```

(Cuando el asistente me ha preguntado por el dominio de la máquina, he introducido el DNS público de la instancia AWS:

ec2-3-230-189-112.compute-1.amazonaws.com. He escogido la opción de generar un certificado autofirmado).

3. Ahora la herramienta ya es totalmente funcional y los usuarios pueden entrar en la sala virtual vía navegador para realizar llamadas seguras de alta calidad en red interna.

2.4 Comprovaciones d'amplada de banda

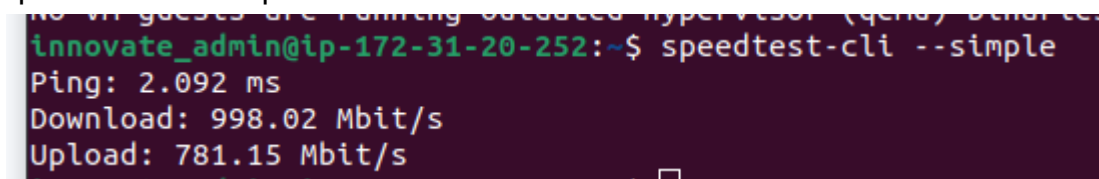
Para garantizar que la infraestructura desplegada es capaz de soportar el consumo de los servicios multimedia sin sufrir degradación alguna, he realizado las medidas de rendimiento de la red de AWS de la siguiente manera:

1. Instalación de la herramienta de test:

```
sudo apt install speedtest-cli -y
```

2. Ejecución de la prueba de rendimiento:

```
speedtest-cli --simple
```

A terminal window with a dark purple background. The prompt is 'innovate_admin@ip-172-31-20-252:~\$'. The command 'speedtest-cli --simple' has been executed. The output shows: 'Ping: 2.092 ms', 'Download: 998.02 Mbit/s', and 'Upload: 781.15 Mbit/s'.

```
innovate_admin@ip-172-31-20-252:~$ speedtest-cli --simple
Ping: 2.092 ms
Download: 998.02 Mbit/s
Upload: 781.15 Mbit/s
```

2.4.1 Análisis de los Resultados y Relación con los Servicios

Una vez ejecutada la prueba de rendimiento en el Servidor 1, hemos obtenido unos resultados que superan las expectativas iniciales para una infraestructura de tipo medio. A continuación, analizo cada métrica según la prueba real:

- **Latencia (2.092 ms):** Este valor es excepcionalmente bajo. En el ámbito de los servicios multimedia, una latencia inferior a 20 ms se considera "tiempo real". Con 2.092 ms, nuestro sistema de

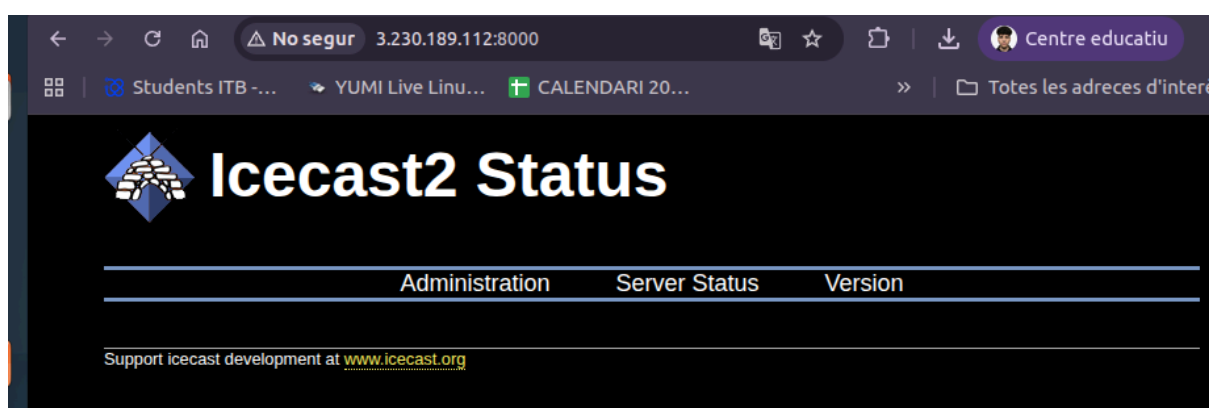
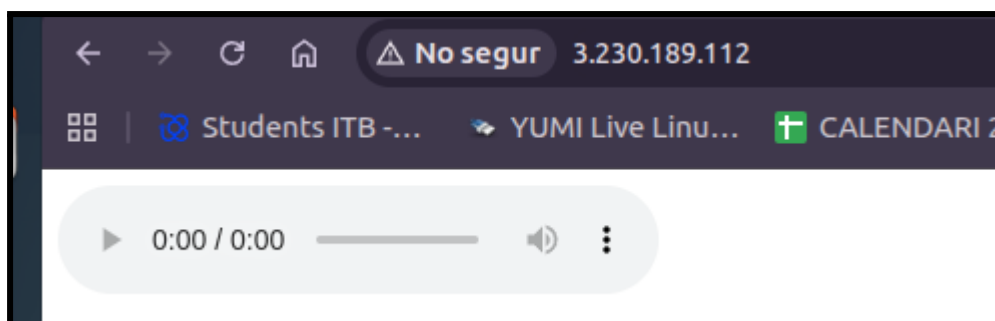
videoconferencia WebRTC (Jitsi) funcionará sin ningún retraso (lag) perceptible, permitiendo una comunicación fluida entre los trabajadores de Innovate Tech. Es un dato clave para evitar que la voz y el vídeo se desincronicen.

- **Ancho de banda de Bajada (998.02 Mbit/s):** Estamos rozando el límite teórico de 1 Gbps. Aunque el servidor actúa principalmente como emisor, esta gran capacidad de descarga nos permite recibir flujos de vídeo de alta calidad de otras fuentes (como cámaras de seguridad o colaboradores externos) y gestionar actualizaciones del sistema sin interrumpir los servicios de streaming que estén activos.
- **Ancho de banda de Subida (781.15 Mbit/s):** Este es el dato más crítico para nuestro proyecto, ya que determina cuántos usuarios pueden ver vídeos o escuchar audio simultáneamente desde nuestro servidor.
 - **Cálculo de capacidad:** Si un vídeo HD (1080p) consume unos 5 Mbps, con nuestros 781 Mbps de subida podríamos dar servicio a más de 150 usuarios simultáneos viendo vídeos distintos sin saturar la red.
 - **Relación con el audio:** El servicio de Icecast2 consume muy poco (aprox. 128-320 Kbps), lo que significa que la radio interna puede funcionar para toda la plantilla sin afectar nada al rendimiento del vídeo o la web.

2.4.2 Clasificación del sistema

De acuerdo con los resultados obtenidos y las necesidades especificadas en el enunciado para Innovate Tech, clasificó el sistema como ACEPTABLE (y altamente eficiente).

La actual infraestructura de AWS proporciona un margen de crecimiento muy amplio. No se prevé ningún cuello de botella ni la necesidad de ampliaciones de hardware o de ancho de banda a corto o medio plazo, incluso si la plantilla se duplicará en el futuro cercano.



2.4.3 Propuestas de Optimización de la Infraestructura

Aunque las pruebas de ancho de banda y rendimiento actuales demuestran que el sistema es aceptable y eficiente para la carga actual de Innovate Tech, se proponen las siguientes mejoras técnicas para garantizar la escalabilidad futura:

1. **Implementación de una CDN (Content Delivery Network):** Para aligerar la carga del servidor principal (EC2), los vídeos de formación pesados y los podcasts estáticos podrían distribuirse a través de una CDN (como Cloudflare o AWS CloudFront). Esto reduciría la latencia y el consumo de ancho de banda de nuestra instancia.
2. **Caché a nivel de Servidor Web:** Configurar un sistema de caché estática en Nginx para que los recursos web (HTML, CSS, imágenes) no tengan que ser procesados en cada petición.
3. **Ajuste de Bitrate Dinámico (ABR):** Para los usuarios con conexiones a internet lentas, se podría implementar un sistema que reduzca automáticamente la calidad del vídeo y del audio (de 128kbps a 64kbps, por ejemplo) para evitar cortes en la reproducción.

2.5 Validación de Servicios y Portal Web Corporativo

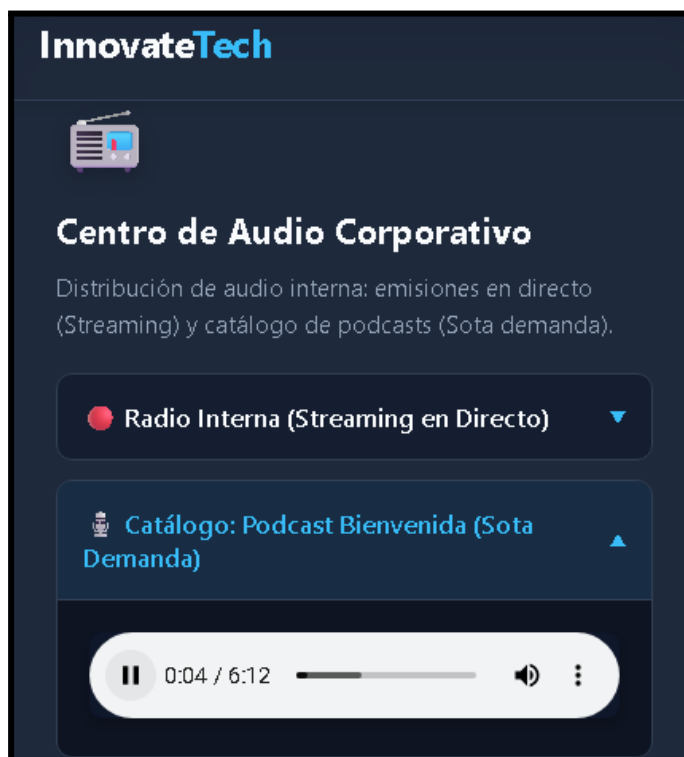
Para centralizar todos los servicios multimedia desplegados en la infraestructura de AWS, se ha desarrollado un portal web corporativo interactivo para *Innovate Tech*. Esta interfaz

sustituye a la página básica de pruebas inicial, ofreciendo un diseño moderno, estructurado y profesional que agrupa el acceso a todas las herramientas en un único punto.

A continuación, se documenta la validación funcional de cada uno de los servicios integrados desde la perspectiva del usuario final (cliente web):

2.5.1 Centro de Audio Corporativo (Streaming y Bajo Demanda)

Se ha verificado el correcto funcionamiento de los reproductores integrados mediante la etiqueta **<audio>** de HTML5. El navegador cliente es capaz de conectarse exitosamente al servidor **Icecast2** para recibir el flujo de streaming en directo, además de reproducir sin cortes el catálogo de podcasts estáticos alojados en el servidor web.



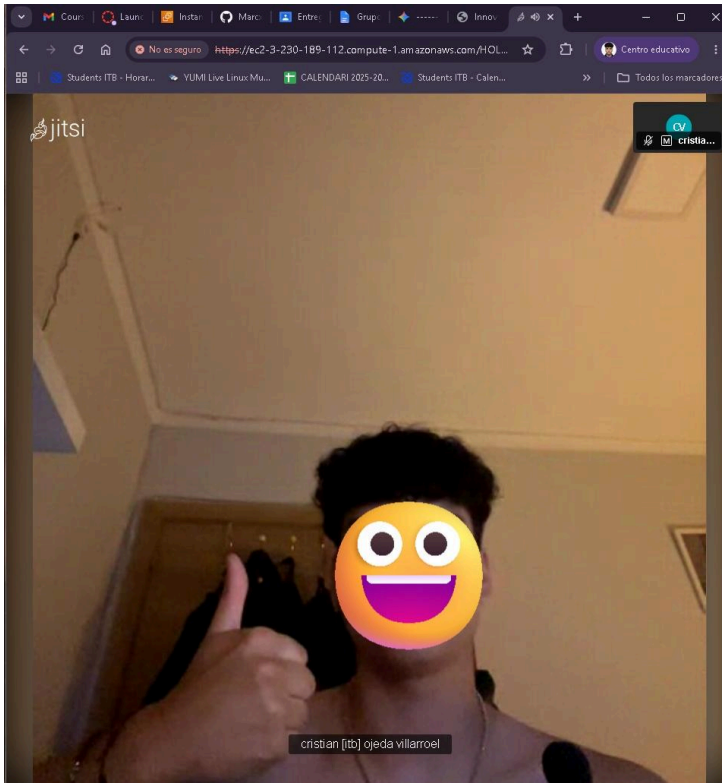
2.5.2 Videoteca de Formación

Se ha validado la distribución de vídeo bajo demanda (VOD). Mediante el uso de la etiqueta **<video>**, el portal es capaz de servir los módulos formativos en formato MP4. Los controles de reproducción nativos responden adecuadamente y la carga del búfer es óptima gracias al ancho de banda de la instancia EC2.



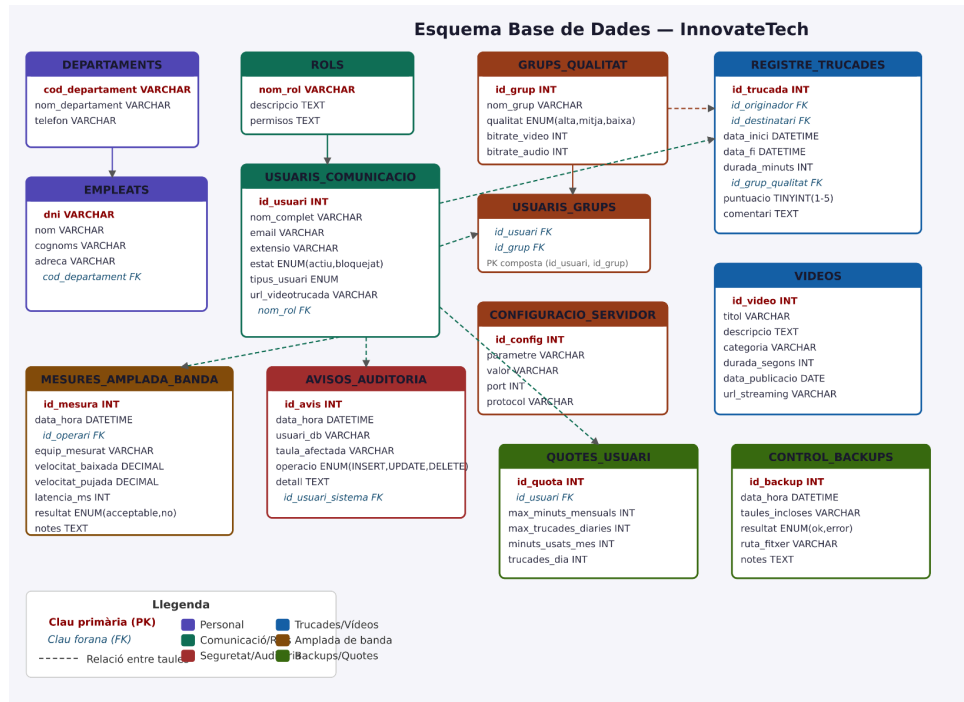
2.5.3 Salas de Videoconferencia (WebRTC)

Finalmente, se ha comprobado la redirección y el acceso al servidor independiente de comunicaciones unificadas (Jitsi). Las pruebas confirman que la conexión WebRTC se establece de forma segura y estable, permitiendo la transmisión bidireccional de cámara y micrófono sin interrupciones, lo que valida la correcta configuración de los puertos UDP (10000) en el Firewall/Security Group de AWS.



3 Diseño e implementación de una base de datos

3.1 Contexto del Proyecto



Gestión del personal - DEPARTAMENTOS y EMPLEADOS (relación 1:N, un empleado pertenece a un solo departamento).

Control de acceso — ROLS define los 4 roles (admin, ventas, administracion, trabajador). USUARIOS_COMUNICACION es la tabla central de los usuarios del sistema, con FK en el rol y campo de estado (activo/bloqueado).

Calidad de servicio — GRUPOS_CALIDAD almacena las configuraciones alta/media/baja. La tabla intermedia USUARIOS_GRUPS gestiona la relación N:M usuario-grupo. CONFIGURACION_SERVIDOR guarda parámetros del servidor de videollamadas.

Registro de actividad — REGISTRO_LLAMADAS registra cada llamada con originador, destinatario, duración y puntuación. VIDEOS contiene el catálogo de streaming con el URL directo.

Seguridad y auditoría — AVISOS_AUDITORIA es la tabla de logs que llenan los triggers. QUOTES_USUARI guarda los límites y el consumo actual para activar los triggers de cuota. CONTROL_BACKUPS registra cada evento de copia de seguridad.

Ancho de banda — MEDIDAS_AMPLADA_BANDA almacena download/upload/latencia con el resultado aceptable/no aceptable y el operario que ha realizado la medida.

3.2 Diseño e Implementación de la Base de Datos

3.2.1 Gestión del Personal y Estructura Organizativa

TABLA departamento:

```
MariaDB [innovatetech]> CREATE TABLE departamentos (  
  ->   id_departamento INT AUTO_INCREMENT PRIMARY KEY,  
  ->   nombre VARCHAR(100) NOT NULL,  
  ->   telefono VARCHAR(20)  
  -> );  
Query OK, 0 rows affected (0.016 sec)
```

```
MariaDB [innovatetech]> SHOW TABLES;  
+-----+  
| Tables_in_innovatetech |  
+-----+  
| departamentos          |  
+-----+  
1 row in set (0.000 sec)  
  
MariaDB [innovatetech]> 
```

TABLA empleados:

```

MariaDB [innovatetech]> CREATE TABLE empleados (
->     dni VARCHAR(9) PRIMARY KEY,
->     nombre VARCHAR(50) NOT NULL,
->     apellidos VARCHAR(100) NOT NULL,
->     direccion TEXT,
->     telefono VARCHAR(20),
->     email VARCHAR(100) UNIQUE,
->     estado ENUM('activo','bloqueado') DEFAULT 'activo',
->     id_departamento INT,
->
->     FOREIGN KEY (id_departamento)
->     REFERENCES departamentos(id_departamento)
-> );
Query OK, 0 rows affected (0.013 sec)

```

```

MariaDB [innovatetech]> SHOW TABLES;
+-----+
| Tables_in_innovatetech |
+-----+
| departamentos          |
| empleados              |
+-----+
2 rows in set (0.000 sec)

MariaDB [innovatetech]> 

```

Probamos de que funciona, insertamos datos:

Departamentos

```

MariaDB [innovatetech]> INSERT INTO departamentos(nombre, telefono)
-> VALUES
-> ('Vendes', '931111111'),
-> ('Administracio', '932222222'),
-> ('Logistica', '933333333'),
-> ('Suport Tecnic', '934444444');
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0

MariaDB [innovatetech]> SELECT * FROM departamentos;
+-----+-----+-----+
| id_departamento | nombre          | telefono |
+-----+-----+-----+
| 1                | Vendes         | 931111111 |
| 2                | Administracio  | 932222222 |
| 3                | Logistica      | 933333333 |
| 4                | Suport Tecnic  | 934444444 |
+-----+-----+-----+
4 rows in set (0.000 sec)

```

Empleados

```
MariaDB [innovatetech]> INSERT INTO empleados
-> VALUES
-> ('12345678A','Juan','Garcia','Barcelona','600111111','juan@innovate.com','activo',1),
-> ('87654321B','Anna','Lopez','Barcelona','600222222','anna@innovate.com','activo',2);
Query OK, 2 rows affected (0.001 sec)
Records: 2 Duplicates: 0 Warnings: 0

MariaDB [innovatetech]> SELECT * FROM empleados;
+-----+-----+-----+-----+-----+-----+-----+-----+
---+
| dni      | nombre | apellidos | direccion | telefono | email              | estado | id_departamen |
to |
+-----+-----+-----+-----+-----+-----+-----+-----+
---+
| 12345678A | Juan   | Garcia    | Barcelona | 600111111 | juan@innovate.com | activo | 1 |
1 |
| 87654321B | Anna   | Lopez     | Barcelona | 600222222 | anna@innovate.com | activo | 2 |
2 |
+-----+-----+-----+-----+-----+-----+-----+-----+
---+
2 rows in set (0.000 sec)

MariaDB [innovatetech]> 
```

Creación de usuario remoto:

```
MariaDB [(none)]> CREATE USER 'admin'@'%' IDENTIFIED BY 'Admin123!';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON innovatetech.* TO 'admin'@'%';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)
```

Ahora que todo va bien podemos seguir creando, creamos la tabla roles:

TABLA roles

```
MariaDB [(none)]> CREATE TABLE roles (
->     id_rol INT AUTO_INCREMENT PRIMARY KEY,
->     nombre VARCHAR(50) UNIQUE NOT NULL
-> );
```

3.2.2 Sistema de Comunicació Interna (Videotrucades i Streaming)

1. usuarios_comunicacion

```
MariaDB [innovatetech]> CREATE TABLE usuarios_comunicacion (  
->     id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
->     dni_empleado VARCHAR(9),  
->     extension_llamada VARCHAR(20) UNIQUE,  
->     correo VARCHAR(100) UNIQUE,  
->     enlace_videollamada TEXT,  
->     tipo_usuario ENUM('interno','cliente'),  
->     estado ENUM('activo','bloqueado') DEFAULT 'activo',  
->     id_rol INT,  
->     FOREIGN KEY (dni_empleado)  
->     REFERENCES empleados(dni),  
->     FOREIGN KEY (id_rol)  
->     REFERENCES roles(id_rol)  
-> );  
Query OK, 0 rows affected (0.028 sec)
```

2. calidad_streaming

```
MariaDB [innovatetech]> CREATE TABLE calidad_streaming (  
->     id_calidad INT AUTO_INCREMENT PRIMARY KEY,  
->     nombre VARCHAR(50),  
->     resolucion VARCHAR(20),  
->     bitrate VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.010 sec)
```

3. configuracion_servidor

- protocolos
- puertos
- parámetros

```
MariaDB [innovatetech]> CREATE TABLE configuracion_servidor (  
->     id_config INT AUTO_INCREMENT PRIMARY KEY,  
->     protocolo VARCHAR(20),  
->     puerto INT,  
->     parametros TEXT  
-> );  
Query OK, 0 rows affected (0.008 sec)
```

4. llamadas

```
MariaDB [innovatetech]> CREATE TABLE llamadas (  
->     id_llamada INT AUTO_INCREMENT PRIMARY KEY,  
->     usuario_origen INT,  
->     usuario_destino INT,  
->     fecha_inicio DATETIME,  
->     fecha_fin DATETIME,  
->     duracion INT,  
->     id_calidad INT,  
->     puntuacion INT,  
->     comentario TEXT,  
->     FOREIGN KEY (usuario_origen)  
->     REFERENCES usuarios_comunicacion(id_usuario),  
->     FOREIGN KEY (usuario_destino)  
->     REFERENCES usuarios_comunicacion(id_usuario),  
->     FOREIGN KEY (id_calidad)  
->     REFERENCES calidad_streaming(id_calidad)  
-> );  
Query OK, 0 rows affected (0.017 sec)
```

5. videos

```
MariaDB [innovatetech]> CREATE TABLE videos (  
->     id_video INT AUTO_INCREMENT PRIMARY KEY,  
->     titulo VARCHAR(200),  
->     descripcion TEXT,  
->     categoria VARCHAR(100),  
->     duracion INT,  
->     fecha_publicacion DATE,  
->     url_streaming TEXT  
-> );  
Query OK, 0 rows affected (0.011 sec)
```

6. mediciones_red

```
MariaDB [innovatetech]> CREATE TABLE mediciones_red (  
->     id_medicion INT AUTO_INCREMENT PRIMARY KEY,  
->     fecha_hora DATETIME,  
->     usuario_equipo VARCHAR(100),  
->     velocidad_bajada DECIMAL(10,2),  
->     velocidad_subida DECIMAL(10,2),  
->     latencia DECIMAL(10,2),  
->     resultado ENUM('acceptable','no acceptable'),  
->     operario VARCHAR(100),  
->     observaciones TEXT  
-> );  
Query OK, 0 rows affected (0.008 sec)
```

Verificamos que están todas la tablas creadas:

```
MariaDB [innovatetech]> SHOW TABLES;
+-----+
| Tables_in_innovatetech |
+-----+
| calidad_streaming      |
| configuracion_servidor |
| departamentos         |
| empleados             |
| llamadas              |
| mediciones_red        |
| roles                 |
| usuarios_comunicacion |
| videos                |
+-----+
9 rows in set (0.001 sec)
```

Ahora para comprobar que todo está bien, insertamos datos de prueba.

1. INSERTAR ROLES

```
MariaDB [innovatetech]> INSERT INTO roles(nombre)
-> VALUES
-> ('admin'),
-> ('vendes'),
-> ('administracio'),
-> ('treballador');
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM roles;
+-----+-----+
| id_rol | nombre      |
+-----+-----+
|      1 | admin       |
|      3 | administracio |
|      4 | treballador  |
|      2 | vendes      |
+-----+-----+
4 rows in set (0.000 sec)
```

2. INSERTAR CALIDADES

```
MariaDB [innovatetech]> INSERT INTO calidad_streaming
-> (nombre,resolucion,bitrate)
-> VALUES
-> ('alta','1080p','6000k'),
-> ('media','720p','3000k'),
-> ('baja','480p','1000k');
Query OK, 3 rows affected (0.009 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM calidad_streaming;
+-----+-----+-----+-----+
| id_calidad | nombre | resolucion | bitrate |
+-----+-----+-----+-----+
|          1 | alta   | 1080p      | 6000k   |
|          2 | media  | 720p       | 3000k   |
|          3 | baja   | 480p       | 1000k   |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

3. INSERTAR CONFIG SERVIDOR

```
MariaDB [innovatetech]> INSERT INTO configuracion_servidor
-> (protocolo, puerto, parametros)
-> VALUES
-> ('WebRTC', 443, 'Servidor principal videollamadas'),
-> ('RTMP', 1935, 'Streaming video'),
-> ('HLS', 8080, 'Streaming adaptativo');
Query OK, 3 rows affected (0.002 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM configuracion_servidor;
+-----+-----+-----+-----+
| id_config | protocolo | puerto | parametros |
+-----+-----+-----+-----+
|          1 | WebRTC    | 443    | Servidor principal videollamadas |
|          2 | RTMP      | 1935   | Streaming video |
|          3 | HLS       | 8080   | Streaming adaptativo |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

4. INSERTAR USUARIOS COMUNICACION

```

MariaDB [innovatetech]> INSERT INTO usuarios_comunicacion
-> (dni_empleado, extension_llamada,
-> correo, enlace_videollamada,
-> tipo_usuario, estado, id_rol)
->
-> VALUES
->
-> ('12345678A',
-> '1001',
-> 'juan@innovate.com',
-> 'https://meet.innovate.com/juan',
-> 'interno',
-> 'activo',
-> 1),
->
-> ('87654321B',
-> '1002',
-> 'anna@innovate.com',
-> 'https://meet.innovate.com/anna',
-> 'interno',
-> 'activo',
-> 2),
->
-> ('45678912C',
-> '1003',
-> 'marc@innovate.com',
-> 'https://meet.innovate.com/marc',
-> 'interno',
-> 'activo',
-> 4);

```

Query OK, 3 rows affected (0.003 sec)
Records: 3 Duplicates: 0 Warnings: 0

```

MariaDB [innovatetech]> SELECT * FROM usuarios_comunicacion;

```

```

+-----+-----+-----+-----+-----+-----+
| id_usuario | dni_empleado | extension_llamada | correo | enlace_videollamada |
| tipo_usuario | estado | id_rol |
+-----+-----+-----+-----+-----+
| 4 | 12345678A | 1001 | | |
| interno | activo | 1 | | |
| 5 | 87654321B | 1002 | | |
| interno | activo | 2 | | |
| 6 | 45678912C | 1003 | | |
| interno | activo | 4 | | |
+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

```

5. INSERTAR VIDEOS

```
MariaDB [innovatetech]> INSERT INTO videos
-> (titulo, descripcion, categoria,
-> duracion, fecha_publicacion,
-> url_streaming)
->
-> VALUES
->
-> ('Tutorial AWS',
-> 'Configuracion EC2',
-> 'Tecnologia',
-> 120,
-> '2025-05-10',
-> 'https://stream.innovate.com/aws'),
->
-> ('Curso Linux',
-> 'Administracion Ubuntu',
-> 'Sistemas',
-> 90,
-> '2025-05-12',
-> 'https://stream.innovate.com/linux');
Query OK, 2 rows affected (0.003 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM videos;
+-----+-----+-----+-----+-----+-----+-----+
| id_video | titulo      | descripcion      | categoria | duracion | fecha_publicacion | url_streaming |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Tutorial AWS | Configuracion EC2 | Tecnologia | 120 | 2025-05-10 | https://stream.innovate.com/aws |
| 2 | Curso Linux | Administracion Ubuntu | Sistemas | 90 | 2025-05-12 | https://stream.innovate.com/linux |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

6. INSERTAR LLAMADAS

```
MariaDB [innovatetech]> INSERT INTO llamadas
-> (usuario_origen,
-> usuario_destino,
-> fecha_inicio,
-> fecha_fin,
-> duracion,
-> id_calidad,
-> puntuacion,
-> comentario)
->
-> VALUES
->
-> (4,
-> 5,
-> '2025-05-20 10:00:00',
-> '2025-05-20 10:30:00',
-> 30,
-> 1,
-> 5,
-> 'Buena calidad'),
->
-> (5,
-> 6,
-> '2025-05-20 11:00:00',
-> '2025-05-20 11:20:00',
-> 20,
-> 2,
-> 4,
-> 'Estable');
Query OK, 2 rows affected (0.001 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM llamadas;
+-----+-----+-----+-----+-----+-----+
| id_llamada | usuario_origen | usuario_destino | fecha_inicio | fecha_fin | duracion |
| id_calidad | puntuacion | comentario |
+-----+-----+-----+-----+-----+-----+
| 4 | 5 | 2025-05-20 10:00:00 | 2025-05-20 10:30:00 | 30 | 1 | Buena calidad |
| 5 | 6 | 2025-05-20 11:00:00 | 2025-05-20 11:20:00 | 20 | 2 | Estable |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

7. INSERTAR MEDICIONES RED

```
MariaDB [innovatetech]> INSERT INTO mediciones_red
-> (fecha_hora,
-> usuario_equipo,
-> velocidad_bajada,
-> velocidad_subida,
-> latencia,
-> resultado,
-> operario,
-> observaciones)
->
-> VALUES
->
-> (NOW(),
-> 'Servidor AWS',
-> 150.50,
-> 80.25,
-> 12.30,
-> 'acceptable',
-> 'Juan',
-> 'Conexion estable'),
->
-> (NOW(),
-> 'Cliente Oficina',
-> 60.20,
-> 25.10,
-> 35.00,
-> 'acceptable',
-> 'Anna',
-> 'Latencia moderada');
Query OK, 2 rows affected (0.002 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
MariaDB [innovatetech]> SELECT * FROM mediciones_red;
+-----+-----+-----+-----+-----+-----+
| id_medicion | fecha_hora          | usuario_equipo | velocidad_bajada | velocidad_subida | latencia |
| resultado  | operario | observaciones |
+-----+-----+-----+-----+-----+-----+
| 1 | 2026-05-20 07:05:07 | Servidor AWS | 150.50 | 80.25 | 12.30 |
| acceptable | Juan | Conexion estable |
| 2 | 2026-05-20 07:05:07 | Cliente Oficina | 60.20 | 25.10 | 35.00 |
| acceptable | Anna | Latencia moderada |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

3.3. Gestió d'Usuaris, Rols i Permisos

PASO 1 — CREAR ROLES SQL

```
MariaDB [innovatetech]> CREATE ROLE admin;
Query OK, 0 rows affected (0.006 sec)

MariaDB [innovatetech]> CREATE ROLE vendas;
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]> CREATE ROLE administracio;
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]> CREATE ROLE treballador;
Query OK, 0 rows affected (0.001 sec)
```

Estos roles permiten gestionar permisos reales sobre las tablas.

2. Asignación de permisos

Rol admin

Tiene acceso total a toda la base de datos.

```
MariaDB [innovatetech]> GRANT ALL PRIVILEGES
-> ON innovatetech.*
-> TO admin;
Query OK, 0 rows affected (0.003 sec)
```

Rol vendas

Puede gestionar llamadas y consultar vídeos.

```
MariaDB [innovatetech]> GRANT SELECT, INSERT, UPDATE
-> ON innovatetech.empleados
-> TO administracio;
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]> GRANT SELECT
-> ON innovatetech.departamentos
-> TO administracio;
Query OK, 0 rows affected (0.001 sec)
```

Rol administracio

Puede gestionar empleados y departamentos.

```
MariaDB [innovatetech]> GRANT SELECT, INSERT, UPDATE
-> ON innovatetech.empleados
-> TO administracio;
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]>
MariaDB [innovatetech]> GRANT SELECT
-> ON innovatetech.departamentos
-> TO administracio;
Query OK, 0 rows affected (0.001 sec)
```

Rol treballador

Solo puede consultar información.

```
MariaDB [innovatetech]> GRANT SELECT
-> ON innovatetech.videos
-> TO treballador;
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]> GRANT SELECT
-> ON innovatetech.llamadas
-> TO treballador;
Query OK, 0 rows affected (0.001 sec)
```

3. Creación de usuarios MariaDB

Se crearon usuarios reales dentro del servidor MariaDB.

```
MariaDB [innovatetech]> CREATE USER 'jesus_palma'@'%'
-> IDENTIFIED BY 'pirineus';
Query OK, 0 rows affected (0.010 sec)

MariaDB [innovatetech]>
MariaDB [innovatetech]> CREATE USER 'cristian_ojeda'@'%'
-> IDENTIFIED BY 'pirineus';
Query OK, 0 rows affected (0.001 sec)

MariaDB [innovatetech]>
MariaDB [innovatetech]> CREATE USER 'marc_jove'@'%'
-> IDENTIFIED BY 'pirineus';
Query OK, 0 rows affected (0.001 sec)
```

Ahora asignamos roles

Jesús → admin

```
MariaDB [innovatetech]> GRANT admin
-> TO 'jesus_palma'@'%'
Query OK, 0 rows affected (0.001 sec)
```

Cristian → vendas

```
MariaDB [innovatetech]> GRANT vendas
-> TO 'cristian_ojeda'@'%'
Query OK, 0 rows affected (0.001 sec)
```

Marc → treballador

```
MariaDB [innovatetech]> GRANT treballador  
-> TO 'marc_jove'@'%';  
Query OK, 0 rows affected (0.001 sec)
```

Activar rol por defecto

```
MariaDB [innovatetech]> SET DEFAULT ROLE admin FOR 'jesus_palma'@'%';  
Query OK, 0 rows affected (0.001 sec)  
  
MariaDB [innovatetech]> SET DEFAULT ROLE vendes FOR 'cristian_ojeda'@'%';  
Query OK, 0 rows affected (0.001 sec)  
  
MariaDB [innovatetech]> SET DEFAULT ROLE treballador FOR 'marc_jove'@'%';  
Query OK, 0 rows affected (0.001 sec)  
  
MariaDB [innovatetech]> □
```

Y Comprobamos que los roles estén bien definidos:

Jesús

```
MariaDB [innovatetech]> SHOW GRANTS FOR 'jesus_palma'@'%';  
+-----+  
| Grants for jesus_palma@%  
|  
+-----+  
| GRANT `admin` TO `jesus_palma`@`%`  
|  
| GRANT USAGE ON *.* TO `jesus_palma`@`%` IDENTIFIED BY PASSWORD '*24B53A8AD41E57CFE01A2E5E7873FA539F3AF8B0' |  
| SET DEFAULT ROLE `admin` FOR `jesus_palma`@`%`  
|  
+-----+  
3 rows in set (0.001 sec)
```

Cristian

```

MariaDB [innovatetech]> SHOW GRANTS FOR 'cristian_ojeda'@'%';
+-----+
| Grants for cristian_ojeda@%
|
+-----+
| GRANT `vendes` TO `cristian_ojeda`@`%`
|
| GRANT USAGE ON *.* TO `cristian_ojeda`@`%` IDENTIFIED BY PASSWORD '*24B53A8AD41E57CFE01A2E5E7873FA539F3AF8B0' |
| SET DEFAULT ROLE `vendes` FOR `cristian_ojeda`@`%`
|
+-----+
3 rows in set (0.000 sec)

```

Marc

```

MariaDB [innovatetech]> SHOW GRANTS FOR 'marc_jove'@'%';
+-----+
| Grants for marc_jove@%
|
+-----+
| GRANT `treballador` TO `marc_jove`@`%`
|
| GRANT USAGE ON *.* TO `marc_jove`@`%` IDENTIFIED BY PASSWORD '*24B53A8AD41E57CFE01A2E5E7873FA539F3AF8B0' |
| SET DEFAULT ROLE `treballador` FOR `marc_jove`@`%`
|
+-----+
3 rows in set (0.001 sec)

```

3.3.2 Script de Creació Automatitzada d'Usuaris

SCRIPT COMPLETO

```
GNU nano 7.2                                crear_usuarios.sh *
#!/bin/bash

USUARIO=$1
ROL=$2
PASSWORD="pirineus"

SQLFILE="usuarios_creados.sql"

if [ -z "$USUARIO" ] || [ -z "$ROL" ]; then
    echo "Uso: ./crear_usuarios.sh usuario rol"
    exit 1
fi

case $ROL in
    admin|vendes|administracio|treballador)
        ;;
    *)
        echo "Error: rol no válido"
        exit 1
        ;;
esac

echo "CREATE USER '$USUARIO'@%' IDENTIFIED BY '$PASSWORD';" >> $SQLFILE

echo "GRANT $ROL TO '$USUARIO'@%';" >> $SQLFILE

echo "SET DEFAULT ROLE $ROL FOR '$USUARIO'@%';" >> $SQLFILE

if [ "$ROL" = "admin" ]; then
    echo "GRANT FILE ON *.* TO '$USUARIO'@%';" >> $SQLFILE
fi

echo "FLUSH PRIVILEGES;" >> $SQLFILE

echo "Usuario preparado correctamente"
echo "SQL generado en $SQLFILE"
```

Y automáticamente:

- crea usuario
- asigna contraseña
- asigna rol
- hace GRANT
- genera SQL

Dar permisos al script:

```
innovate_admin@ip-172-31-22-189:~$ sudo chmod +x crear_usuarios.sh
innovate_admin@ip-172-31-22-189:~$
```

EJEMPLO DE USO

```
innovate_admin@ip-172-31-22-189:~$ ./crear_usuarios.sh jesus_palma admin
Usuario preparado correctamente
SQL generado en usuarios_creados.sql
innovate_admin@ip-172-31-22-189:~$
```

Qué generará

Un archivo:

usuarios_creados.sql

con:

```
CREATE USER 'jesus_palma'@'%' IDENTIFIED BY 'pirineus';
GRANT admin TO 'jesus_palma'@'%';
SET DEFAULT ROLE admin FOR 'jesus_palma'@'%';
GRANT FILE ON *.* TO 'jesus_palma'@'%';
FLUSH PRIVILEGES;
```

Cómo ejecutarlo después

```
innovate_admin@ip-172-31-22-189:~$ sudo mysql -u root -p < usuarios_creados.sql
Enter password:
innovate_admin@ip-172-31-22-189:~$
```

Script final — crear_usuarios.sh, es el script final con todo lo que pide la parte del proyecto.

```
GNU nano 7.2                                crear_usuarios.sh
#!/bin/bash

USUARIO=$1
ROL=$2
PASSWORD="pirineus"

SQLFILE="usuarios_creados.sql"

# Comprobación de parámetros
if [ -z "$USUARIO" ] || [ -z "$ROL" ]; then
    echo "Uso: ./crear_usuarios.sh usuario rol"
    exit 1
fi

# Validación de roles
case $ROL in
    admin|vendes|administracio|treballador)
        ;;
    *)
        echo "Error: rol no válido"
        exit 1
        ;;
esac

# Comprobar si el usuario ya existe
if sudo mysql -e "SELECT User FROM mysql.user WHERE User='$USUARIO'" | grep $USUARIO > /dev/null
then
    echo "Error: el usuario ya existe"
    exit 1
fi

# Comentario en el SQL
echo "-- Usuario generado automáticamente" >> $SQLFILE

# CREATE USER
echo "CREATE USER '$USUARIO'@%' IDENTIFIED BY '$PASSWORD';" >> $SQLFILE

# GRANT DEL ROL
echo "GRANT $ROL TO '$USUARIO'@%';" >> $SQLFILE

# DEFAULT ROLE
echo "SET DEFAULT ROLE $ROL FOR '$USUARIO'@%';" >> $SQLFILE

# Permiso especial FILE para admin
if [ "$ROL" = "admin" ]; then
    echo "GRANT FILE ON *.* TO '$USUARIO'@%';" >> $SQLFILE
fi

# Aplicar privilegios
echo "FLUSH PRIVILEGES;" >> $SQLFILE

[ Wrote 53 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

Qué mejoras se añadieron

1. Control de usuario duplicado

```
if sudo mysql -e "SELECT User FROM mysql.user WHERE User='$USUARIO'" | grep
$USUARIO > /dev/null
then
    echo "Error: el usuario ya existe"
    exit 1
fi
```

¿Qué hace?

Antes de generar el SQL:
busca el usuario en MariaDB
si existe:
muestra error
detiene el script

¿Por qué se añadió?

Porque la práctica pide:
“gestionar casos de error”
Y uno de los errores más comunes es:
intentar crear un usuario ya existente.

2. Comentario automático en el SQL

`echo "-- Usuario generado automáticamente" >> $SQLFILE`

¿Qué hace?

Añade un comentario al archivo .sql.

Ejemplo:

-- Usuario generado automáticamente

¿Por qué se añadió?

Para:
mejorar la legibilidad
hacer el SQL más profesional
facilitar revisión posterior

3.3.3 Triggers per al Control d'Accés i Auditoria

PASO 1 — Crear tabla avisos

Esta tabla guardará:

- accesos no autorizados
- bloqueos
- intentos prohibidos
- avisos automáticos

CREA LA TABLA

```
MariaDB [(none)]> USE innovatetech
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [innovatetech]> CREATE TABLE avisos_auditoria (
  ->
  ->     id_aviso INT AUTO_INCREMENT PRIMARY KEY,
  ->
  ->     usuario VARCHAR(100),
  ->
  ->     tabla_afectada VARCHAR(100),
  ->
  ->     operacion VARCHAR(50),
  ->
  ->     descripcion TEXT,
  ->
  ->     fecha_hora DATETIME DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.018 sec)
```

Qué hace

Cada vez que pase algo sospechoso:
guardaremos un registro aquí.

PASO 2 — Trigger cuota mensual

Objetivo:

- limitar minutos mensuales
- impedir llamadas si supera el límite

Un ejemplo sería 300 minutos al mes.

CREA EL TRIGGER

```
MariaDB [innovatetech]> CREATE TRIGGER cuota_mensual
-> BEFORE INSERT ON llamadas
-> FOR EACH ROW
->
-> BEGIN
->
->     DECLARE total_minutos INT;
->
->     SELECT IFNULL(SUM(duracion),0)
->     INTO total_minutos
->     FROM llamadas
->     WHERE usuario_origen = NEW.usuario_origen
->     AND MONTH(fecha_inicio) = MONTH(NOW());
->
->     IF total_minutos + NEW.duracion > 300 THEN
->
->         INSERT INTO avisos_auditoria
->         (usuario, tabla_afectada, operacion, descripcion)
->
->         VALUES
->         (
->             NEW.usuario_origen,
->             'llamadas',
->             'INSERT',
->             'Límite mensual de minutos superado'
->         );
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Quota mensual superada';
->
->     END IF;
->
-> END//
Query OK, 0 rows affected (0.010 sec)
```

PASO 4 — Usuarios bloqueados

Objetivo:

- impedir llamadas
- si usuario está bloqueado

CREA EL TRIGGER

```
MariaDB [innovatetech]> CREATE TRIGGER usuario_bloqueado
-> BEFORE INSERT ON llamadas
-> FOR EACH ROW
->
-> BEGIN
->
->     DECLARE estado_usuario VARCHAR(20);
->
->     SELECT estado
->     INTO estado_usuario
->     FROM usuarios_comunicacion
->     WHERE id_usuario = NEW.usuario_origen;
->
->     IF estado_usuario = 'bloqueado' THEN
->
->         INSERT INTO avisos_auditoria
->         (usuario, tabla_afectada, operacion, descripcion)
->
->         VALUES
->         (
->             NEW.usuario_origen,
->             'llamadas',
->             'INSERT',
->             'Usuario bloqueado intentó realizar llamada'
->         );
->
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Usuario bloqueado';
->
->     END IF;
->
-> END//
Query OK, 0 rows affected (0.019 sec)
```

PASO 5 — Trigger de auditoría

Ahora haremos auditoría automática.

IDEA

Si un usuario intenta modificar algo restringido:
registramos el intento.

EJEMPLO SIMPLE

Vamos a auditar modificaciones en roles.

CREA EL TRIGGER

```
MariaDB [innovatetech]> CREATE TRIGGER auditoria_rols
-> BEFORE UPDATE ON roles
-> FOR EACH ROW
->
-> BEGIN
->
->     INSERT INTO avisos_auditoria
->     (usuario, tabla_afectada, operacion, descripcion)
->
->     VALUES
->     (
->         CURRENT_USER(),
->         'roles',
->         'UPDATE',
->         'Intento de modificación de roles'
->     );
->
-> END//
Query OK, 0 rows affected (0.013 sec)
```

PASO 6 — PROBAR

Ver triggers

```
MariaDB [innovatetech]> SELECT TRIGGER_NAME
-> FROM information_schema.TRIGGERS
-> WHERE TRIGGER_SCHEMA='innovatetech';
+-----+
| TRIGGER_NAME |
+-----+
| cuota_mensual |
| usuario_bloqueado |
| auditoria_rols |
+-----+
3 rows in set (0.003 sec)

MariaDB [innovatetech]> 
```

(Con esto salen los triggers de una manera más ordenada porque con SHOW TRIGGERS; sale todo desordenado y no es práctico ponerlo en el documento.)

Y probamos el trigger:

```
MariaDB [innovatetech]> INSERT INTO llamadas
-> (
-> usuario_origen,
-> usuario_destino,
-> fecha_inicio,
-> fecha_fin,
-> duracion,
-> id_calidad,
-> puntuacion,
-> comentario
-> )
->
-> VALUES
-> (
-> 4,
-> 5,
-> NOW(),
-> NOW(),
-> 10,
-> 1,
-> 5,
-> 'Prueba bloqueo'
-> );
ERROR 1644 (45000): Usuario bloqueado
MariaDB [innovatetech]> 
```

Y sale el aviso:

```
MariaDB [innovatetech]> SELECT * FROM avisos_auditoria;
+-----+-----+-----+-----+-----+-----+
| id_aviso | usuario | tabla_afectada | operacion | descripcion | fecha_hora |
+-----+-----+-----+-----+-----+-----+
| 2 | 4 | llamadas | INSERT | Usuario bloqueado intentó realizar llamada | 2026-05-21 07:36:01 |
| 3 | 4 | llamadas | INSERT | Usuario bloqueado intentó realizar llamada | 2026-05-21 07:36:13 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

El trigger funciona PERFECTAMENTE

Porque:

- detecta usuario bloqueado
- impide llamada
- registra auditoría

Para el bot de telegram no tengo capturas de pantalla ya que lo he hecho casi todo desde el móvil pero dejo los pasos detallados.

PASO 1 — Crear bot Telegram

En Telegram busca:

@BotFather

PASO 2 — Crear bot

Escribe:

/newbot

PASO 3 — Nombre del bot

InnovateTechAlertBot

PASO 4 — Username

innovate_alert_bot

PASO 5 — Guardar TOKEN

BotFather te dará un token, En mi caso es:

8680942666:AAEUW7KITHNjaJaZQNvzpuHJ6rDp70jiBTw

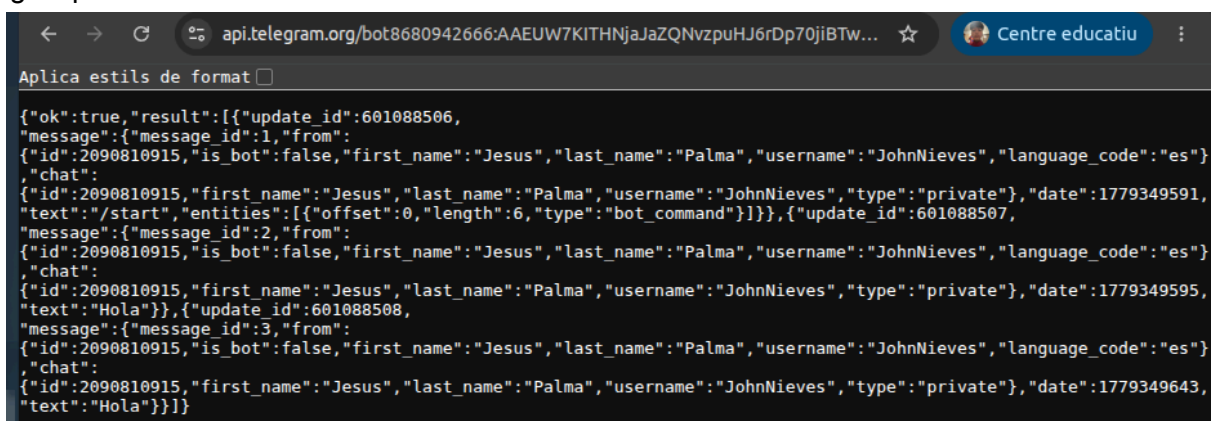
PASO 6 — Obtener tu chat_id

En Telegram escribe cualquier mensaje a tu bot:

hola

PASO 7 — Abrir en navegador

<https://api.telegram.org/bot8680942666:AAEUW7KITHNjaJaZQNvzpuHJ6rDp70jiBTw/getUpdates>



```
{
  "ok": true,
  "result": [
    {
      "update_id": 601088506,
      "message": {
        "message_id": 1,
        "from": {
          "id": 2090810915,
          "is_bot": false,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "language_code": "es"
        },
        "chat": {
          "id": 2090810915,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "type": "private",
          "date": 1779349591,
          "text": "/start",
          "entities": [
            {
              "offset": 0,
              "length": 6,
              "type": "bot_command"
            }
          ]
        },
        "message_id": 2,
        "from": {
          "id": 2090810915,
          "is_bot": false,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "language_code": "es"
        },
        "chat": {
          "id": 2090810915,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "type": "private",
          "date": 1779349595,
          "text": "Hola"
        },
        "message_id": 3,
        "from": {
          "id": 2090810915,
          "is_bot": false,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "language_code": "es"
        },
        "chat": {
          "id": 2090810915,
          "first_name": "Jesus",
          "last_name": "Palma",
          "username": "JohnNieves",
          "type": "private",
          "date": 1779349643,
          "text": "Hola"
        }
      }
    }
  ]
}
```

PASO 8 — Crear script alerta

nano alerta_telegram.sh

```
GNU nano 7.2 alerta_telegram.sh
#!/bin/bash

TOKEN="8680942666:AAEUW7KITHNjaJaZQNvzpuHJ6rDp70jiBTw"
CHAT_ID="2090810915"

MENSAJE="⚠️Alerta InnovateTech: intento sospechoso detectado"

curl -s -X POST https://api.telegram.org/bot$TOKEN/sendMessage \
-d chat_id=$CHAT_ID \
-d text="$MENSAJE"
```

PASO 10 — Ejecutar

```
ch: intento sospechoso detectado"}}innovate_admin@ip-172-31-22-189:~$ sudo nano alerta_telegram.sh
innovate_admin@ip-172-31-22-189:~$ ./alerta_telegram.sh
{"ok":true,"result":{"message_id":6,"from":{"id":8680942666,"is_bot":true,"first_name":"Innovate_aler
t_bot","username":"Innovate_alertbot"},"chat":{"id":2090810915,"first_name":"Jesus","last_name":"Palm
a","username":"JohnNieves","type":"private"},"date":1779350358,"text":"\u26a0\ufe0f Alerta InnovateTe
ch: intento sospechoso detectado"}}innovate_admin@ip-172-31-22-189:~$
```

Comprobamos en el bot:



3.3.4 Events Periòdics (Còpies de Seguretat)

PASO 1 — Tabla control backups

```
MariaDB [innovatetech]> CREATE TABLE control_backups (  
->   
->     id_backup INT AUTO_INCREMENT PRIMARY KEY,  
->   
->     fecha_hora DATETIME DEFAULT CURRENT_TIMESTAMP,  
->   
->     tablas_incluidas TEXT,  
->   
->     resultado VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.011 sec)
```

Qué hace

Guardarà:

- cuándo se hizo backup
- qué tablas
- si fue correcto

PASO 2 — Activar scheduler

MariaDB NO ejecuta eventos automáticamente si esto está apagado.

```
MariaDB [innovatetech]> SET GLOBAL event_scheduler = ON;  
Query OK, 0 rows affected (0.000 sec)
```

Comprobamos:

```
MariaDB [innovatetech]> SHOW VARIABLES LIKE 'event_scheduler';  
+-----+-----+  
| Variable_name | Value |  
+-----+-----+  
| event_scheduler | ON    |  
+-----+-----+  
1 row in set (0.001 sec)
```

Tiene que salir "ON"

PASO 3 — Crear carpeta backup

Crea carpeta

```
innovate_admin@ip-172-31-22-189:~$ mkdir backups
```

Le damos permisos:

```
innovate_admin@ip-172-31-22-189:~$ sudo chmod 777 backups
```

Mi ruta actual es:

```
innovate_admin@ip-172-31-22-189:~$ pwd
/home/innovate_admin
```

Y la ruta final tiene que ser “/home/innovate_admin/backups/”

PASO 4 — Crear EVENT automático

```
MariaDB [innovatetech]> CREATE EVENT backup_diario
->
-> ON SCHEDULE EVERY 1 DAY
->
-> DO
->
-> BEGIN
->
->     SELECT *
->     FROM empleados
->     INTO OUTFILE '/home/innovate_admin/backups/empleados_backup.txt';
->
->     SELECT *
->     FROM llamadas
->     INTO OUTFILE '/home/innovate_admin/backups/llamadas_backup.txt';
->
->     INSERT INTO control_backups
->     (
->         tablas_incluidas,
->         resultado
->     )
->
->     VALUES
->     (
->         'empleados, llamadas',
->         'Backup correcto'
->     );
->
-> END//
Query OK, 0 rows affected, 2 warnings (0.002 sec)
```

Qué hace

- Cada día automáticamente:
- exporta empleados
- exporta llamadas
- guarda archivos backup
- registra ejecución

PASO 5 — Comprobar EVENT

Ver eventos:

```
MariaDB [innovatetech]> SHOW EVENTS;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Db      | Name      | Definer      | Time zone | Type      | Execute at | Interval value |
| Interval field | Starts      | Ends      | Status  | Originator | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+-----+-----+
| innovatetech | backup_diario | root@localhost | SYSTEM   | RECURRING | NULL       | 1              |
| DAY         | 2026-05-21 08:40:17 | NULL      | ENABLED  | 1         | utf8mb3    |                | utf8mb
3_general_ci | utf8mb4_general_ci |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

Ver tabla control (Hay que esperar un día, Lo he cambiado para que salga cada minuto para poderlo poner en la practica)

```
MariaDB [innovatetech]> SELECT * FROM control_backups;
+-----+-----+-----+-----+
| id_backup | fecha_hora      | tablas_incluidas | resultado      |
+-----+-----+-----+-----+
| 1         | 2026-05-26 06:13:39 | empleados, llamadas | Backup correcto |
+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

```
MariaDB [innovatetech]> █
```

Breve explicación ya que fueron muchos comandos:

Qué implementamos

“Se creó un sistema automático de copias de seguridad usando EVENTS de MariaDB.”

Cómo funciona

“El scheduler ejecuta automáticamente un backup periódico exportando tablas críticas mediante INTO OUTFILE.”

Qué tablas incluye

“Actualmente se respaldan las tablas empleados y llamadas.”

Auditoría

“Cada ejecución queda registrada en la tabla control_backups indicando fecha, tablas incluidas y resultado del backup.”

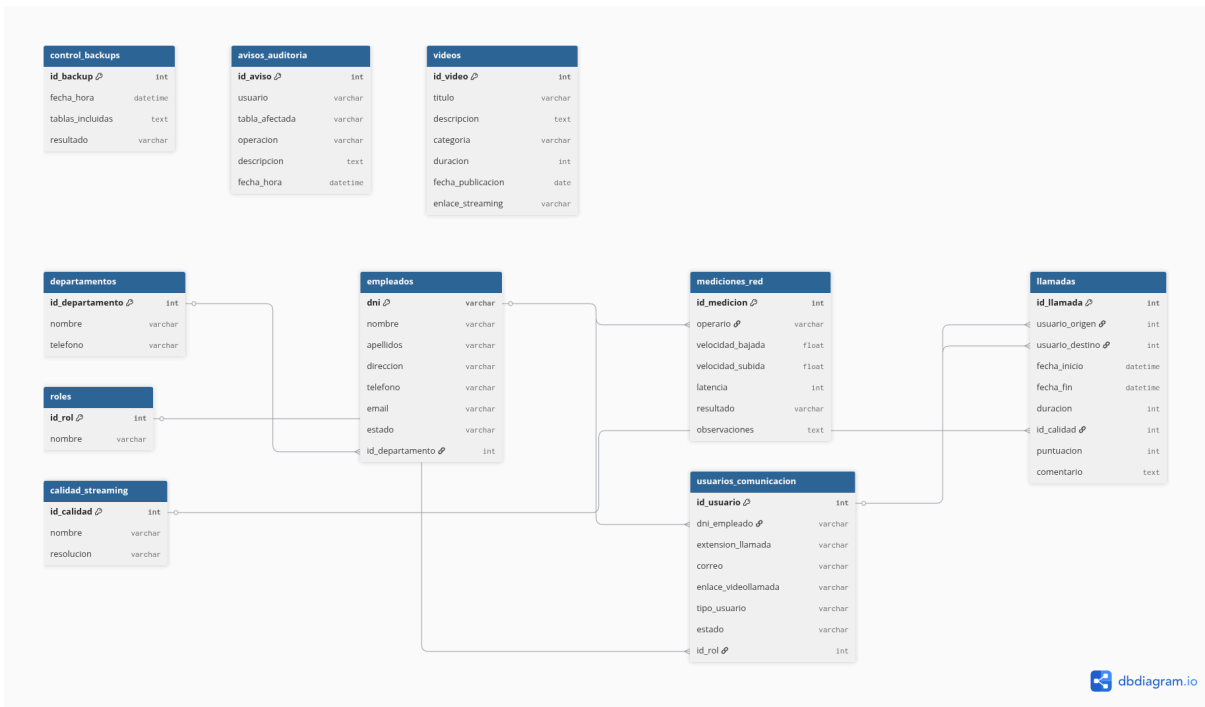
Justificación periodicidad

“Se eligió una periodicidad diaria porque la plataforma gestiona llamadas y usuarios continuamente y se minimiza la pérdida de datos.”

3.4. Disseny Entitat-Relació i Model Relacional

A partir de los requisitos funcionales del sistema InnovateTech, se diseñó un modelo Entidad-Relación que representa la gestión del personal, videollamadas, streaming, calidad de servicio, auditoría y copias de seguridad.

Diagrama Entidad-Relación



El diagrama muestra las relaciones entre las distintas entidades de la base de datos, incluyendo claves primarias (PK) y claves foráneas (FK).

Modelo relacional

Tabla: empleados

Campo	Tipo	Restricción
DNI	VARCHAR	PK
nombre	VARCHAR	NOT NULL
apellidos	VARCHAR	NOT NULL
direccion	VARCHAR	NOT NULL
teléfono	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
estado	VARCHAR	NOT NULL
id_departamento	INT	FK

Tabla: roles

Campo	Tipo	Restricción
id_rol	INT	PK
nombre	VARCHAR	UNIQUE, NOT NULL

Tabla: usuarios_comunicacion

Campo	Tipo	Restricción
id_usuario	VARCHAR	PK
dni_empleado	VARCHAR	FK
extension_llamada	VARCHAR	UNIQUE
correo	VARCHAR	UNIQUE
enlace_videollamada	VARCHAR	NOT NULL
tipo_usuario	VARCHAR	NOT NULL
estado	VARCHAR	NOT NULL
id_rol	INT	FK

Tabla: llamadas

Campo	Tipo	Restricción
id_llamada	INT	PK
usuario_origen	INT	FK
usuario_destino	INT	FK
fecha_inicio	DATETIME	NOT NULL
fecha_fin	DATETIME	NOT NULL
duracion	INT	NOT NULL
id_calidad	INT	FK
puntuacion	INT	
comentario	TEXT	

Tabla: departamentos

Campo	Tipo	Restricción
id_departamento	INT	PK
nombre	VARCHAR	UNIQUE, NOT NULL
telefono	VARCHAR	NOT NULL

Tabla: calidad_streaming

Campo	Tipo	Restricción
id_calidad	INT	PK
nombre	VARCHAR	NOT NULL
resolucion	VARCHAR	NOT NULL

Tabla: videos

Campo	Tipo	Restricción
id_video	INT	PK
titulo	VARCHAR	NOT NULL
descripcion	TEXT	
categoria	VARCHAR	NOT NULL
duracion	INT	NOT NULL
fecha_publicacion	DATE	DATE
enlace_streaming	VARCHAR	NOT NULL

Tabla: mediciones_red

Campo	Tipo	Restricción
id_medicion	INT	PK
operario	VARCHAR	FK
velocidad_bajada	FLOAT	NOT NULL
velocidad_subida	FLOAT	NOT NULL
latencia	INT	NOT NULL
resultado	VARCHAR	NOT NULL
observaciones	TEXT	

Tabla: avisos_auditoria

Campo	Tipo	Restricción
id_aviso	INT	PK
usuario	VARCHAR	NOT NULL
tabla_afectada	VARCHAR	NOT NULL
operacion	VARCHAR	NOT NULL
descripcion	TEXT	
fecha_hora	DATETIME	NOT NULL

Tabla: control_backups

Campo	Tipo	Restricción
id_backup	INT	PK
fecha_hora	DATETIME	NOT NULL
tablas_incluidas	TEXT	NOT NULL
resultado	VARCHAR	NOT NULL

Implementación

La práctica pide demostrar:

- tablas creadas

```
MariaDB [innovatetech]> SHOW TABLES;
+-----+
| Tables_in_innovatetech |
+-----+
| avisos_auditoria       |
| calidad_streaming      |
| configuracion_servidor |
| control_backups        |
| departamentos          |
| empleados              |
| llamadas               |
| mediciones_red         |
| roles                  |
| usuarios_comunicacion  |
| videos                 |
+-----+
11 rows in set (0.000 sec)

MariaDB [innovatetech]> 
```

- datos insertados

Empleados:

```
MariaDB [innovatetech]> SELECT
-> dni AS DNI,
-> nombre AS Nombre,
-> apellidos AS Apellidos,
-> telefono AS Telefono
-> FROM empleados;
+-----+-----+-----+-----+
| DNI      | Nombre | Apellidos | Telefono |
+-----+-----+-----+-----+
| 12345678A | Juan   | Garcia    | 600111111 |
| 45678912C | Marc   | Perez     | 600333333 |
| 87654321B | Anna   | Lopez     | 600222222 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

usuarios_comunicacion:

```

MariaDB [innovatetech]> SELECT
  -> id_usuario AS ID,
  -> dni_empleado AS DNI_Empleado,
  -> extension_llamada AS Extension,
  -> correo AS Correo,
  -> tipo_usuario AS Tipo,
  -> estado AS Estado,
  -> id_rol AS Rol
  -> FROM usuarios_comunicacion;
+-----+-----+-----+-----+-----+-----+-----+
| ID | DNI_Empleado | Extension | Correo           | Tipo   | Estado   | Rol |
+-----+-----+-----+-----+-----+-----+-----+
| 4 | 12345678A   | 1001     | juan@innovate.com | interno | bloqueado | 1 |
| 5 | 87654321B   | 1002     | anna@innovate.com | interno | activo    | 2 |
| 6 | 45678912C   | 1003     | marc@innovate.com | interno | activo    | 4 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

```

llamadas:

```

MariaDB [innovatetech]> SELECT
  -> id_llamada AS ID,
  -> usuario_origen AS Origen,
  -> usuario_destino AS Destino,
  -> fecha_inicio AS Inicio,
  -> fecha_fin AS Fin,
  -> duracion AS Duracion_Min,
  -> puntuacion AS Puntuacion
  -> FROM llamadas;
+-----+-----+-----+-----+-----+-----+-----+
| ID | Origen | Destino | Inicio           | Fin           | Duracion_Min | Puntuacion |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | 4 | 5 | 2025-05-20 10:00:00 | 2025-05-20 10:30:00 | 30 | 5 |
| 4 | 5 | 6 | 2025-05-20 11:00:00 | 2025-05-20 11:20:00 | 20 | 4 |
| 5 | 4 | 5 | 2026-05-21 07:36:01 | 2026-05-21 07:36:01 | 10 | 5 |
| 6 | 4 | 5 | 2026-05-21 07:36:13 | 2026-05-21 07:36:13 | 10 | 5 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.000 sec)

```

control_backups:

```

MariaDB [innovatetech]> SELECT
  -> id_backup AS ID,
  -> fecha_hora AS Fecha,
  -> tablas_incluidas AS Tablas,
  -> resultado AS Resultado
  -> FROM control_backups;
+-----+-----+-----+-----+
| ID | Fecha           | Tablas           | Resultado       |
+-----+-----+-----+-----+
| 1 | 2026-05-26 06:13:39 | empleados, llamadas | Backup correcto |
+-----+-----+-----+-----+
1 row in set (0.000 sec)

```

avisos_auditoria:

```

MariaDB [innovatetech]> SELECT
  -> id_aviso AS ID,
  -> usuario AS Usuario,
  -> tabla_afectada AS Tabla,
  -> operacion AS Operacion,
  -> fecha_hora AS Fecha
  -> FROM avisos_auditoria;
+-----+-----+-----+-----+-----+
| ID | Usuario | Tabla   | Operacion | Fecha                |
+-----+-----+-----+-----+-----+
| 2  | 4       | llamadas | INSERT    | 2026-05-21 07:36:01 |
| 3  | 4       | llamadas | INSERT    | 2026-05-21 07:36:13 |
+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)

```

3.5 Integración de Servicios y Conexión Remota

Para que el S1 (Web) se conecte al S4 (MariaDB), debes realizar y documentar estos 3 pasos:

1. **Configurar MariaDB en S4 para permitir conexiones remotas:**
 - Editar el archivo: **sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf**
 - Cambiar **bind-address = 127.0.0.1** por **bind-address = 0.0.0.0**.
 - Reiniciar: **sudo systemctl restart mariadb**

```

GNU nano 7.2 /etc/mysql/mariadb.conf.d/50-server.cnf
#
# These groups are read by MariaDB server.
# Use it for options that only the server (but not clients) should see
#
# this is read by the standalone daemon and embedded servers
[server]
#
# this is only for the mysqld standalone daemon
[mysqld]
#
# * Basic Settings
#
#user                    = mysql
pid-file                 = /run/mysqld/mysqld.pid
basedir                  = /usr
#datadir                 = /var/lib/mysql
#tmpdir                   = /tmp
#
# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address              = 0.0.0.0

```

2. **Crear el usuario con permisos remotos en S4:**

- Entra en MariaDB y ejecuta:

```
CREATE USER 'web_user1'@'3.230.189.112' IDENTIFIED BY 'pirineus';
```

```
GRANT ALL PRIVILEGES ON innovate_tech.* TO 'web_user1'@'3.230.189.112';
```

FLUSH PRIVILEGES;

```
MariaDB [(none)]> CREATE USER 'web_user1'@'3.230.189.112'  
-> IDENTIFIED BY 'pirineus';  
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES  
-> ON innovatetech.*  
-> TO 'web_user1'@'3.230.189.112';  
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> CREATE USER 'web_user1'@'3.230.189.112'  
-> IDENTIFIED BY 'pirineus';  
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES  
-> ON innovatetech.*  
-> TO 'web_user1'@'3.230.189.112';  
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.001 sec)
```

3. Prueba de conexión desde el Servidor 1 (Documentación):

- Instala el cliente en S1: **sudo apt install mariadb-client**
- **Comando de comprobación (Pon captura de esto):**

```
sudo mysql -u web_user1 -h 54.146.49.143 -p
```

(Hay que usar la ip publica del servidor base de datos)

```
innovate_admin@ip-172-31-20-252:~$ sudo mysql -u web_user1 -h 54.146.49.143 -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 35
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| innovatetech |
+-----+
2 rows in set (0.003 sec)

MariaDB [(none)]> 
```

5. Puntos de Control Transversal

5.1 Evaluación de la Seguridad Lógica de los Datos (RA5)

Para proteger la información de *Innovate Tech* alojada en la nube, se han aplicado estrictas medidas de seguridad lógica en los servidores:

- **Firewall perimetral (AWS Security Groups):** Se ha aplicado el principio de mínimo privilegio. Solo se han abierto los puertos estrictamente necesarios: 80 (HTTP), 443 (HTTPS), 8000 (Icecast) y 10000 UDP (WebRTC para Jitsi). El resto del tráfico entrante está bloqueado.
- **Acceso SSH Seguro:** El acceso remoto de administración a los servidores Linux se realiza exclusivamente mediante claves criptográficas (pares de claves de AWS), deshabilitando el acceso por contraseña estándar para evitar ataques de fuerza bruta.
- **Seguridad en Bases de Datos:** El servidor MariaDB cuenta con usuarios específicos con permisos limitados (sentencias **GRANT**). El usuario web no tiene acceso de administrador (**root**), protegiendo la base de datos ante posibles inyecciones SQL desde el portal multimedia.

5.2 Uso de Herramientas Colaborativas (RA6)

Durante el desarrollo de este proyecto, el Grupo 5 ha operado simulando un entorno empresarial real, utilizando herramientas de trabajo colaborativo para garantizar la sincronización:

- **Documentación y Archivos:** Se ha utilizado Google Docs para la redacción simultánea del proyecto, permitiendo control de versiones en tiempo real.
- **Comunicación Diaria:** Se han establecido canales de mensajería instantánea (/Discord) para la toma de decisiones rápidas y la resolución de bloqueos técnicos durante la configuración de los servidores o de documentación.
- **Gestión de Tareas:** La división del trabajo (configuración de red, bases de datos, despliegue web) se ha gestionado de forma ágil, asignando responsabilidades claras a cada miembro del equipo para cumplir con los plazos de entrega.

5.2 Monitorización y Logs Centralizados

Para asegurar la alta disponibilidad, hemos implementado un sistema de logs y monitorización centralizada con Netdata. El Servidor de Logs recolecta en tiempo real métricas de CPU, RAM y red de todo el CPD mediante streaming.

1. Instalar Netdata en todos los servidores

Es un script automático que configura todo:

```
wget -O /tmp/netdata-kickstart.sh https://my-netdata.io/kickstart.sh && sudo sh /tmp/netdata-kickstart.sh --non-interactive
```

2. Configurar el "Streaming" (Centralización)

El servidor 2 (Logs) será el "padre" y los demás servidores los "hijos"

- Servidor de Logs (Padre):
Generar una clave única (API KEY):
uuidgen
(Copiarlo)

Editar el archivo de streaming:

```
sudo nano /etc/netdata/stream.conf
```

Y pegar esto en el fichero junto con la API creada:

```
[dd5bf6a3-e9af-4116-9740-ad269ddb7664]  
enabled = yes  
default history = 3600
```

Reiniciar: **sudo systemctl restart netdata**

- **Servidores 1, 2 y 4 (Hijos):**

Editar el archivo de streaming:

sudo nano /etc/netdata/stream.conf

Configurar el envío hacia la IP Publica del Servidor de Logs:

[stream]

enabled = yes

destination = 54.175.41.208:19999

api key = dd5bf6a3-e9af-4116-9740-ad269ddb7664

3. Integrarlo en el Portal Web

```
<section id="monitoring">
```

```
  <h2>Panel de Monitorización en Tiempo Real (CPD Innovate Tech)</h2>
```

```
  <p>Estado de salud de los servidores centralizado mediante Netdata.</p>
```

```
  <iframe src="http://54.175.41.208:19999"
```

```
    width="100%"
```

```
    height="600"
```

```
    style="border:none;">
```

```
  </iframe>
```

```
</section>
```

```
</section>
<section id="monitoring">
  <h2>Panel de Monitorización en Tiempo Real (CPD Innovate Tech)</h2>
  <p>Estado de salud de los servidores centralizado mediante Netdata.</p>

  <iframe src="http://54.175.41.208:19999"
    width="100%"
    height="600"
    style="border:none;">
  </iframe>
</section>
main>
```

¡IMPORTANTE!: Para que esto funcione en nuestra página web, debemos ir a AWS Security Groups del Servidor de Logs y abrir el puerto 19999 (TCP) para "Anywhere".

Comprobación en la página web:



Entrar como anónimo debajo de “sign-in”

Darle a “Nodes” y ver que tenemos los servidores por IP (Faltan dos servidores pendientes)

